Do the Numbers Hold on Real Hardware? A Reproducibility and Measurement Audit of a Low-Cost Hybrid ECC Authentication Protocol for IoT

ON

Submitted in partial fulfillment of the requirements of
the degree of

Bachelor of Engineering

(Information Technology)

By

Jay Kerkar – Roll No. 16
Chaitanya Kelkar – Roll No. 15
Chirag Poornamath – Roll No. 26
Yash Kolekar – Roll No. 17

Under the guidance of

Dr. Shanta Sondur

Department of Information Technology
VIVEKANAND EDUCATION SOCIETY'S INSTITUTE OF TECHNOLOGY,
Chembur, Mumbai 400074
(An Autonomous Institute, Affiliated to University of Mumbai)

AY (2026-27)

Vivekanand Education Society's
Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai, Approved by AICTE & Recognised by Govt. of Maharashtra)
NAAC accredited with 'A' grade

Certificate

This is to certify that project entitled

"Do the Numbers Hold on Real Hardware? A Reproducibility and Measurement Audit of a Low-Cost Hybrid ECC Authentication Protocol for IoT"

Group Members Names

Jay Kerkar (Roll No. 16)
Chaitanya Kelkar (Roll No. 15)
Chirag Poornamath (Roll No. 26)
Yash Kolekar (Roll No. 17)

In partial fulfilment of the Fourth Year (Sem. VI–Sem. VII), Bachelor of Engineering (B.E.) Degree in Information Technology at the University of Mumbai

Academic Year 2026-27

Dr. Shanta Sondur **Project Mentor**	**External Examiner**
Dr. (Mrs.) Shalu Chopra **H.O.D**	**Dr. (Mrs.) J. M. Nair** **Principal**

Date: / /2026
Place:
VESIT, Chembur

College Seal

DECLARATION

We hereby declare that the project report entitled **Do the Numbers Hold on Real Hardware? A Reproducibility and Measurement Audit of a Low-Cost Hybrid ECC Authentication Protocol for IoT** is our original work carried out as part of the Semester VII Capstone Project. The work has been completed under the guidance of Dr. Shanta Sondur. All sources of information used in this report have been appropriately acknowledged and cited. This report has not been submitted, in whole or in part, for any other degree or qualification.

Name: Jay Kerkar	Name: Chaitanya Kelkar
Roll No.: 16	Roll No.: 15
Signature: _____	Signature: _____
Name: Chirag Poornamath	Name: Yash Kolekar
Roll No.: 26	Roll No.: 17
Signature: _____	Signature: _____

ACKNOWLEDGEMENT

The project report on **Do the Numbers Hold on Real Hardware? A Reproducibility and Measurement Audit of a Low-Cost Hybrid ECC Authentication Protocol for IoT** is the outcome of the guidance, moral support and devotion bestowed on our group throughout our work. We express our sincere gratitude to Dr. (Mrs.) Shalu Chopra, Head of the Department of Information Technology **[VERIFY: name/designation]**, to **[MISSING: Deputy HOD name and designation, if to be included]**, and to our project guide Dr. Shanta Sondur, for their valuable inputs, consistent guidance and support. We also thank Vivekanand Education Society's Institute of Technology for providing a stimulating atmosphere and a conducive work environment, including institutional access to the IEEE Xplore digital library through which the base paper for this project was obtained.

[MISSING: Any other individuals or organisations the group wishes to thank.]

ABSTRACT

Internet of Things (IoT) devices and servers must authenticate each other before exchanging data over shared wireless channels, and the authentication must be cheap enough for constrained devices. Al-Rasheed *et al.* (IEEE Transactions on Consumer Electronics, 2026) propose a hybrid protocol that uses elliptic curve cryptography only in an offline registration phase and a two-message symmetric-key handshake online. They report about 17%, 21% and 12% lower communication, processing and storage cost than prior schemes, based on simulation and assumed per-operation times. This project implements the protocol and tests those claims. A Python reference implementation follows the paper's nonce-based model, because its narrative algorithm is replayable. It passes 108 tests with 97% coverage, including 10,000 key-agreement trials and eight adversarial cases. A C port of the device side runs on an ESP8266 microcontroller and interoperates with the Python server, costing 5.26 ms of computation per handshake. Measuring the underlying primitives with one benchmark on the ESP8266, an AWS Graviton2 core and a laptop shows that an ECC scalar multiplication takes 276 ms on the ESP8266, sixteen times the assumed value, while symmetric operations are five to twelve times faster than assumed. With these measured constants, the protocol is 60 to 2,500 times cheaper than the ECC-based schemes it is compared with. However, it costs more than the verified hash-only schemes once the session-key derivation required by its own equations is counted. A consistency audit finds that the paper's headline percentages cannot be derived from its own tables or figures, and its simulated processing figure cannot be reproduced from published information. The protocol also lacks forward secrecy, and its server-side pseudonym lookup grows with the number of devices. A field test on two cloud instances found three implementation weaknesses that a deployment would need to fix.

Keywords: Internet of Things, Mutual Authentication, Elliptic Curve Cryptography, Reproducibility, Microcontroller Benchmarking, Lightweight Security

CONTENTS

Certificate i

Declaration ii

Acknowledgement iii

Abstract iv

List of Figures vii

List of Tables viii

List of Abbreviations ix

1 Introduction 2

1.1 Introduction . . . 2
1.2 Problem Definition . . . 2
1.3 Objectives . . . 3
1.4 Motivation . . . 4
1.5 Scope of the Work . . . 4
1.6 Feasibility Study . . . 5
1.7 Organization of the report . . . 5

2 Literature Survey 8

2.1 Introduction . . . 8
2.2 Review of Literature Survey . . . 8
2.3 Research gaps . . . 14
2.4 Conclusion . . . 15

3 Proposed Methodology 17

3.1 Introduction . . . 17
3.2 Requirement Gathering . . . 17
3.2.1 Functional Requirements . . . 18
3.2.2 Non Functional Requirements . . . 19
3.2.3 User Requirements . . . 19
3.3 Proposed Algorithms . . . 20
3.4 Gantt chart for Timeline . . . 22
3.5 Architectural Diagram . . . 23

3.5.1 UML Diagrams 24
3.5.2 Block Diagram 27
3.5.3 Data Flow Diagram 27
3.5.4 Timeline Chart 28
3.6 Hardware Requirements 28
3.7 Software Requirements 30
3.8 Cost Estimation 31
3.9 Conclusion 31

4 Implementation & Results **34**
4.1 Introduction 34
4.2 Implementation 34
4.3 Results of Implementation 38
4.4 Result Analysis 41
4.5 Observation/Remarks 44
4.6 Conclusion 45

5 Conclusion **48**
5.1 Limitations 48
5.2 Conclusion 48
5.3 Future Scope 49

References **50**

Appendix **53**
1. Paper Details 53
2. Competition certificates 53

LIST OF FIGURES

3.1 Overall project workflow 17
3.2 Gantt chart of project activities (from repository commit dates) 23
3.3 Layered architecture of the implementation 24
3.4 Use-case diagram 24
3.5 Class diagram of the protocol core 25
3.6 Sequence diagram: offline registration and two-message mutual authentication 26
3.7 Block diagram of the system components 27
3.8 Data flow diagram, level 0 27
3.9 Data flow diagram, level 1 28
3.10 Project milestone timeline (2026) 28

4.1 Communication cost: measured (proposed) vs. paper-reported baselines (Table IV) 40
4.2 OMNeT++ sensitivity analysis of the base paper's Table II setting with measured ESP8266 constants (team's model; mean of 5 seeds) 43
4.3 Processing cost (device + server, log scale) of the proposed scheme and the comparator schemes under the base paper's constants, the comparator papers' own constants, and measured ESP8266 constants (team's replication) 44

LIST OF TABLES

2.1 Comparative Summary of Literature Survey 12
2.2 Further schemes cited by the base paper (secondary-source descriptions) 13
2.3 Research Gap Analysis 14

3.1 Functional requirements (from the PRD) and implementation status . . 18
3.2 Non-functional requirements (from the PRD) and observed status 19
3.3 User requirements (from the PRD) 19
3.4 Hardware used for development and measurement 29
3.5 Software requirements 30
3.6 Cost estimation 31

4.1 Implementation components (lines include comments) 35
4.2 Paper symbol to code traceability 37
4.3 Line coverage of `protocol/` and `crypto/` 38
4.4 Adversarial test cases (PRD Section 7.2), all passing 38
4.5 AWS t4g.nano field test (device and server on separate instances; 2,000 handshakes, 0 failures, 2,000 unique session keys) 39
4.6 Measured message sizes per session 40
4.7 Measured processing time per session on the Linux laptop (ms) 40
4.8 Paper constants vs. measured medians (GCM: 56-byte AES-256-GCM including key setup, `aes_big`/`aes_ct`) 41
4.9 Consistency audit of the base paper 42
4.10 Hash-only rows of Table V cheaper than the proposed scheme (device + server; GCM, cached key, `aes_big`; re-run with `eq22_variants.py`) 44

LIST OF ABBREVIATIONS

AAD	Additional Authenticated Data
AEAD	Authenticated Encryption with Associated Data
AES	Advanced Encryption Standard
CI	Continuous Integration
CLI	Command-Line Interface
CSPRNG	Cryptographically Secure Pseudo-Random Number Generator
DFD	Data Flow Diagram
ECC	Elliptic Curve Cryptography
ECDH	Elliptic Curve Diffie–Hellman
GCM	Galois/Counter Mode
HKDF	HMAC-based Key Derivation Function
IoT	Internet of Things
KDF	Key Derivation Function
MAC	Message Authentication Code (or Media Access Control address, by context)
MCU	Microcontroller Unit
NIST	National Institute of Standards and Technology
PID	Pseudonymous Identifier
PoC	Proof of Concept
PPT	Probabilistic Polynomial Time
PRD	Product Requirements Document
SHA	Secure Hash Algorithm
TA	Trusted Authority
TCP	Transmission Control Protocol
UML	Unified Modeling Language

CHAPTER: 1

INTRODUCTION

CHAPTER 1

INTRODUCTION

1.1 Introduction

The Internet of Things (IoT) connects sensors, actuators, appliances and other embedded systems that sense their environment, exchange data with nearby devices and servers, and act on what they receive [1]. These devices usually communicate over shared wireless channels, so an adversary on the same channel can eavesdrop, replay earlier messages, inject forged ones, or impersonate a legitimate server. Before a device and a server exchange application data, each must therefore confirm that the other is a trusted participant. This is called *mutual authentication*.

Lightweight authentication schemes for IoT are usually compared by counting the cryptographic operations each party performs and multiplying by an assumed time per operation. The result of such a comparison then depends on a few constants that are rarely measured on the kind of device the scheme targets [2]. Al-Rasheed *et al.* [1] propose a hybrid scheme in which elliptic curve cryptography (ECC) is used only in an offline registration phase run by a Trusted Authority (TA), and every online authentication uses a two-message, nonce-based symmetric handshake. Using an OMNeT++ simulation and assumed per-operation times, they report about 17%, 21% and 12% lower communication, processing and storage cost than prior schemes.

This project takes that paper as its base. It first turns the protocol into working, tested software: a Python reference implementation and a C port of the device side that runs on an ESP8266 microcontroller. It then uses that software to check the paper's claims: it audits the paper's tables and figures for consistency, measures the cryptographic primitives on real hardware, recomputes the paper's cost model with the measured values, and examines security properties the paper does not discuss. The results were written up by the team as a manuscript [2].

1.2 Problem Definition

At the start of the project the protocol existed only as a paper description that had been validated in simulation. As recorded in the project's requirements document (PRD, v0.1, 25 August 2026), there was no way to:

1. confirm that the protocol can be implemented as specified without hidden ambiguities;
2. verify its claimed security properties with concrete, executable tests; or

3. validate the paper's quantitative claims against an independent, code-based measurement rather than a simulator.

Two further problems emerged while working on the paper. First, its evidence cannot be re-run: the simulation behind its figures has no published code, scenario or data, and its cost tables rely on assumed constants (e.g. 17.1 ms for an ECC scalar multiplication) that were never measured on IoT hardware. Second, the paper describes the online phase twice. The narrative algorithm in Section II reuses a static digest across sessions and is therefore replayable. The mathematical model in Section III uses fresh nonces and is the version the paper's security sketch covers. An implementation must choose one and justify the choice.

These problems matter to security reviewers who need a concrete artefact to audit, to researchers who want to compare against the scheme, and to engineers deciding whether the scheme is worth deploying on constrained devices.

1.3 Objectives

The first group of objectives comes from the PRD (goals G1–G7 and its acceptance criteria). The second group was added when the project was extended into a measurement-based audit; it follows the contributions listed in the team's manuscript [2].

Implementation objectives.

1. Implement offline TA registration, pseudonym derivation and the two-message mutual authentication of Section III as a tested Python library, with at least 90% line coverage on the `protocol/` and `crypto/` packages.
2. Verify session-key agreement (Eq. 3.5) in at least 10,000 randomised trials with zero failures.
3. Provide TA, device and server command-line tools that run a complete registration and authentication on one machine, including a visible, correctly classified rejection of a replayed message.
4. Implement and pass eight adversarial test cases (SEC-01 to SEC-08).
5. Measure communication, processing and storage cost on real code, and complete one authentication in under 5 ms median (excluding network) on a laptop CPU.

Measurement and audit objectives.

6. Port the device side to C and run it on a microcontroller (ESP8266) against the unmodified Python server.
7. Measure the primitives the paper's cost model uses (hash, symmetric encryption, ECC scalar multiplication, KDF) on the ESP8266, a cloud ARM core and a laptop, using one C source on every platform.
8. Audit the paper's tables, equations and figures for internal consistency, and check

its competitor rows against the original papers where available.

9. Recompute the paper's cost model with the measured constants, and state for each claim whether it is supported.
10. Attempt to reproduce the paper's simulated processing figure (Fig. 1), and report honestly what can and cannot be reproduced.

1.4 Motivation

Three observations motivated this work. First, IoT authentication must be cheap: constrained devices cannot afford heavy asymmetric cryptography or multi-message handshakes on every session, and the paper's design targets exactly this constraint. Second, lightweight schemes have a history of being broken after publication. Chang and Le [3] showed that the scheme of Turkanović *et al.* [4] is vulnerable to several attacks, and Jiang *et al.* [5] showed comparable flaws in an earlier three-factor scheme. A protocol that has only been simulated deserves an executable implementation before anyone relies on it. Third, the paper's conclusions depend on assumed constants. If a scalar multiplication or an encryption costs very different amounts on a real microcontroller, the ranking of schemes can change, so the constants themselves have to be measured.

1.5 Scope of the Work

In scope.

- A Python package (`hybrid_ecc_auth`) with cryptographic wrappers, the TA, device and server state machines, a versioned wire format, replay and block-list bookkeeping, an optional login pre-check, command-line demonstrations over TCP, a benchmark harness and a test suite with continuous integration.
- A portable C device core and ESP8266 (NodeMCU) firmware that interoperate with the Python server, plus a C primitive benchmark.
- A two-node field test of the Python implementation on AWS t4g.nano instances.
- A consistency audit of the base paper, a recomputation of its cost model with measured constants, protocol-level checks (forward secrecy, server lookup cost), an attempted reproduction of its Fig. 1, and a replication of its competitor rows and simulation environment, including an OMNeT++ model used as a sensitivity analysis.

Intended users (from the PRD): security reviewers and auditors, applied-cryptography researchers, product-engineering teams evaluating a production version, and firmware teams who may port the protocol.

Out of scope.

- Production hardening: no hardware security module, no formal verification

(ProVerif or Tamarin), no penetration test, no side-channel protection.

- Device mobility and hand-off; devices are treated as static, as in the paper.
- TA-assisted rekeying (Eq. 24 of the paper), which is a stub only.
- Re-implementing the nine competitor protocols; their costs are the paper's operation counts multiplied by measured times.
- 8- and 16-bit microcontrollers, MCUs with cryptographic hardware, and energy measurement.
- Running the Section II ECDH registration exchange; the TA derives secrets locally as in Eq. 8.

1.6 Feasibility Study

Technical feasibility. All primitives the protocol needs (NIST P-256, SHA-256, HKDF-SHA256, AES-256-GCM) are available in the open-source `cryptography` library [6] for Python and in BearSSL [7], which ships with the ESP8266 Arduino core. Feasibility has been demonstrated in practice: the full test suite and the paper-scale benchmark run in seconds on a laptop, the C device core completes handshakes on an ESP8266 over Wi-Fi, and the protocol ran between two cloud instances (Chapter 4).

Operational feasibility. The demonstration needs three terminals on one machine and a single `pip install -e .`. The ESP8266 firmware is built and flashed with PlatformIO, and a native build of the same C core can be checked against the Python server without hardware.

Economic feasibility. All software used is free or open source. Hardware consists of laptops, two NodeMCU ESP8266 boards and short-lived AWS t4g.nano instances, which the AWS demo report prices at US$0.0042 per hour each. The base paper was obtained through the institute's IEEE Xplore access. **[MISSING: Prices of the ESP8266 boards, the AWS bill in rupees, and whether the laptops are personally owned or institute-provided.]**

1.7 Organization of the report

Chapter 2 reviews the literature on lightweight IoT mutual authentication, including the base paper and the comparator papers whose full texts were studied, and sets out the research gaps. Chapter 3 presents the methodology: requirements, the protocol algorithms, the cost-model recomputation, the project timeline, architecture and UML diagrams, hardware and software requirements, and cost estimation. Chapter 4 describes the implementation (Python, C/ESP8266, cloud test, audit and replication tooling) and presents and analyses the results. Chapter 5 gives limitations, conclusions and future scope. The appendix holds publication and competition details.

CHAPTER: 2

LITERATURE SURVEY

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction

This survey examines how existing IoT mutual authentication schemes balance security against communication, computation and storage overhead, and locates the base paper [1] within that landscape. Because this project also audits the base paper's comparison, the survey pays particular attention to the comparator schemes the base paper evaluates: what they actually compute, which time constants their own authors used, and whether the base paper transcribed them correctly. The sources are peer-reviewed journal and conference papers: the base paper, three comparator papers whose full texts were studied (Dhillon and Kalra; Chang and Le; Jiang *et al.*), and further schemes cited by the base paper, which are described from secondary sources only.

2.2 Review of Literature Survey

Paper 1

Title: A Low-Cost Hybrid Elliptic Curve Cryptography Authentication Protocol for Trustworthy Internet of Things Communication
Link: `https://doi.org/10.1109/TCE.2026.3662053`
Authors and year: A. Al-Rasheed, R. Khan, A. Ahmad, A. Saeed, F. Alturise and S. Alkhalaf, 2026 [1].
Summary. The paper addresses mutual authentication between IoT devices and servers over shared channels. A Trusted Authority registers every device and server offline, deriving a per-entity secret $\lambda_X = H_1(ID_X \| s) \bmod n$ and distributing the secrets of trusted peers. Online, a device and server derive a pairwise symmetric root key and authenticate each other with two messages: an encrypted challenge carrying a rotating pseudonym and a fresh nonce, and an encrypted response carrying the server's nonce, after which both derive a session key. The scheme block-lists an entity after three failed attempts and conditions communication on radio coverage. It is evaluated by OMNeT++ simulation and by operation counts multiplied by assumed constants (Table III) against nine prior schemes.
Observations. (i) Online, only a KDF, symmetric encryption and a MAC are used (Eq. 22). (ii) The abstract reports about 17%, 21% and 12% lower communication, processing and storage cost. (iii) The Section II algorithm and the Section III model describe

different protocols; only Section III uses per-session nonces. (iv) The cost constants are assumed, not measured. (v) The paper contains internal inconsistencies: Table IV lists the proposed scheme with 4 messages and 1,024 bits, the text says 2 messages and 264 bits; Section V names OMNeT++ while the conclusion names SUMO; Table II lists a 160 km/h speed for supposedly static devices; and several comparator labels in Tables IV–V point to the wrong entries in its own reference list. (vi) No simulation code, scenario or data is published.

Inference drawn. The Section III model is implementable and its security sketch can be turned into tests, but the quantitative claims rest on evidence that cannot be re-run. The paper therefore needs both an implementation and an independent, measurement-based check. It is the base paper of this project.

Paper 2

Title: A lightweight biometrics based remote user authentication scheme for IoT services

Link: `https://doi.org/10.1016/j.jisa.2017.01.003`

Authors and year: P. K. Dhillon and S. Kalra, 2017 [8].

Summary. A multi-factor (password, smart device, biometric) remote user authentication and key agreement scheme. A user registers through a gateway node and then connects directly to a sensor node. Only one-way hashes, perceptual hashes and XOR operations are used.

Observations. Security is argued against multiple attacks and verified formally with AVISPA. Computation is very low because there are no encryption or ECC operations. The paper gives no per-operation time constants of its own, only that a hash value is 128 bits. Its Table 3 tabulates Turkanović *et al.* at 7/5/7 hash operations for user, sensor node and gateway. The authors leave a testbed evaluation on real devices to future work.

Inference drawn. A hash-only scheme can be cheaper to compute than a scheme that uses symmetric encryption and a KDF. This paper is one of the two independent sources that confirm the base paper's hash-only comparator rows, and it also shows the base paper's Turkanović row has device and server counts swapped.

Paper 3

Title: A Provably Secure, Efficient, and Flexible Authentication Scheme for Ad hoc Wireless Sensor Networks

Link: `https://doi.org/10.1109/TWC.2015.2473165`

Authors and year: C.-C. Chang and H.-D. Le, 2016 [3].

Summary. The paper cryptanalyses the smart-card scheme of Turkanović *et al.* and proposes two replacements: P1, which uses only hash and XOR operations, and P2, an

ECC-based variant with perfect forward secrecy.

Observations. Turkanović *et al.*'s scheme is shown vulnerable to impersonation with node capture, stolen smart cards, sensor-node spoofing and stolen verifiers, and to lack backward secrecy. Security of P1 and P2 is proven in the random oracle model. The authors use $t_h = 0.5$ ms and $t_{mul} = 63.075$ ms. The team's replication found that one printed cell (P2's user cost) does not match the paper's own constants. The base paper's row labelled "Cheng and Le [14]" matches P2 with the user and gateway hash counts exchanged, while its reference [14] points to a different paper (Wang *et al.*, VANETs).

Inference drawn. The paper shows the trade-off between per-session ECC (forward secrecy, higher cost) and hash-only operation. The base scheme avoids per-session ECC entirely, so it cannot provide forward secrecy; this project tested that directly.

Paper 4

Title: Robust extended chaotic maps-based three-factor authentication scheme preserving biometric template privacy

Link: `https://doi.org/10.1007/s11071-015-2467-5`

Authors and year: Q. Jiang, F. Wei, S. Fu, J. Ma, G. Li and A. Alelaiwi, 2016 [5].

Summary. The paper shows that Islam's chaotic-map three-factor scheme has an invalid password check, is vulnerable to impersonation and off-line password guessing when the smart card is lost, and fails to protect the biometric template. It proposes a robust replacement and proves mutual authentication with BAN logic.

Observations. The scheme is *not* hash-only. Its authentication phase costs $7T_H + 4T_C$ and login plus authentication cost $11T_H + 6T_C$, with the authors' own constants $T_H = 0.20$ ms and $T_C = 32.40$ ms. The base paper's Table III sets $T_C = T_H = 0.32$ ms, about 100 times lower. The base paper's row labelled "Jiang et al. [15]" (7/5/10 hash operations) does not match this paper. A 2025 survey [9] lists a different, two-factor scheme by Jiang *et al.* [10] at those counts, which is the likely source.

Inference drawn. The constants a comparison uses can decide its outcome. With its own T_C, this scheme becomes far more expensive than the base scheme, whereas the base paper's constants make it look cheaper. Comparator rows must be traced to their sources before conclusions are drawn from them.

Paper 5

Title: A novel user authentication and key agreement scheme for heterogeneous ad hoc wireless sensor networks, based on the Internet of Things notion

Link: **[MISSING: DOI/URL – not in repo]**

Authors and year: M. Turkanović, B. Brumen and M. Hölbl, 2014 [4].

[VERIFY: The full text of this paper is not in the repository. This entry relies on

the base paper, the team's literature survey, and the two papers that tabulate it (Chang and Le; Dhillon and Kalra). Please read the original and revise, or replace it with a paper whose full text you have.]

Summary. A smart-card-based user authentication and key agreement scheme for heterogeneous ad hoc wireless sensor networks in an IoT setting, using only hash and XOR operations.

Observations. It is the cheapest scheme in the base paper's Table V. Chang and Le and Dhillon and Kalra both tabulate it at 7/5/7 hash operations (user/sensor/gateway); the base paper prints 7/7/5, swapping device and server. Chang and Le later showed it to be insecure, and it provides no forward secrecy.

Inference drawn. The cheapest scheme in a cost table may not be secure, so cost alone cannot rank schemes. Because its operation counts are confirmed by two independent papers, it is the cleanest hash-only comparator for the base scheme.

Table 2.1: Comparative Summary of Literature Survey

Sr. No.	Author(s), Year	Paper Title	Methodology / Technology	Key Observations	Inference Drawn
1	Al-Rasheed *et al.*, 2026	Low-cost hybrid ECC authentication protocol for trustworthy IoT	Offline ECC (TA); online KDF, AES-GCM, MAC; 2 messages; OMNeT++	Claims 17/21/12% savings; assumed constants; inconsistent tables; no published simulation	Implementable; claims need measurement-based checking
2	Dhillon & Kalra, 2017	Lightweight biometrics-based remote user authentication for IoT services	Hash, perceptual hash, XOR; gateway; AVISPA	Very low computation; formally verified; no time constants	Hash-only scheme; confirms Turkanović counts
3	Chang & Le, 2016	Provably secure, efficient, flexible authentication for ad hoc WSNs	Hash/XOR (P1), ECC (P2); random oracle proofs	Breaks Turkanović; P2 has forward secrecy; base paper's "Cheng–Le" row is P2	Base scheme lacks forward secrecy
4	Jiang *et al.*, 2016	Robust extended chaotic maps-based three-factor authentication	Chebyshev chaotic maps, smart card, biometrics; BAN logic	$T_C =$ 32.40 ms, not 0.32 ms; not the base paper's "Jiang" row	Constants decide rankings; trace rows to sources
5	Turkanović *et al.*, 2014	User authentication and key agreement for heterogeneous ad hoc WSNs	Smart card, hash, XOR	Cheapest in Table V; later broken; device/server counts swapped in base paper	Cost alone is not sufficient

The base paper also compares or cites the schemes in Table 2.2. Their descriptions come from the base paper, the team's survey document and the team's manuscript; their full texts are not in the repository.

Table 2.2: Further schemes cited by the base paper (secondary-source descriptions)

Sr. No.	Author(s), Year	Mechanism	Limitation / note
6	Zong *et al.*, 2024 [11]	Blockchain, revocable access control (ReLAC)	Revocation needs key regeneration or re-encryption
7	Nawaz Khan *et al.*, 2024 [12]	Distributed public/private-key scheme	Overly complex for large networks
8	Gong *et al.*, 2023 [13]	Binary symmetric polynomials (LCDMA)	Cross-domain key distribution complexity
9	Alshawish & Al-Haj, 2022 [14]	Hash + AES, six messages	Lengthy exchange
10	Qiu *et al.*, 2025 [15]	Blockchain, medical IoT	Ledger overhead on low-power nodes
11	Yin & Gong, 2024 [16]	Certificateless signatures	Per-session signature overhead
12	Sharma & Dhiman, 2024 [17]	Hash + XOR addressing	Centralised gateway
13	Porambage *et al.*, 2014 [18]	Two-phase ECC authentication	Most expensive row in Table V
14	Sadhukhan *et al.*, 2021 [19]	ECC remote user authentication	Per-session ECC
15	Challa *et al.*, 2017 [20]	Signature-based key establishment	Shares reference [17] with the “Wazid” row in the base paper
16	Fakroon *et al.*, 2020 [21]	Anonymous remote user authentication	Bit total confirmed by [22]
17	Shuai *et al.*, 2019 [23]	Smart-home anonymous authentication, hash + ECC	Likely source of the “Shuai [16]” row, via [22]; base paper adds one device ECC operation
18	Jiang *et al.*, 2015 [10]	Two-factor WSN authentication, hash-only	Likely source of the “Jiang [15]” row, via [9]

2.3 Research gaps

Table 2.3 combines the gaps listed in the team's literature survey document with those found while implementing and auditing the base paper.

Table 2.3: Research Gap Analysis

Sr. No.	Existing Approach / System	Identified Limitation / Research Gap	Proposed Project's Approach
1	Base paper	No implementation; validated only in simulation	Python reference implementation and an ESP8266 C device port that interoperate
2	Base paper, Section II	Static digest reused across sessions; replayable	Implement the Section III nonce-based model; test replay rejection
3	Base paper, Table III	Per-operation times assumed, never measured on IoT hardware	Measure primitives on ESP8266, Graviton2 and a laptop with one C source
4	Base paper, Tables IV–VI and abstract	Headline percentages not derived; tables contradict text	Systematic consistency audit under nine definitions of “reduction”
5	Base paper, Table V	Comparator rows not traceable; session KDF omitted from own count	Check rows against original papers; recompute with four readings (V1–V4)
6	Base paper, Fig. 1	Simulation has no published code, scenario or data	Attempted reproduction; own OMNeT++ model as sensitivity analysis only
7	Base paper, Section III	Epoch handling unspecified; forward secrecy and server lookup cost not discussed	Documented epoch window; tested forward secrecy and lookup scaling
8	Base paper; most comparators	No machine-checked security proof	Executable adversarial tests only; formal verification left as future work
9	Base paper; ReLAC	Static devices; single TA; no lightweight revocation	Not addressed; future work

The central gap is the evidence gap in rows 3–6. The base paper's ranking of schemes depends on per-operation constants that were never measured, on comparator rows that cannot all be traced to their sources, and on a simulation that cannot be re-run. Until these are checked, neither the scheme's advantages nor its disadvantages

are known. An implementation (rows 1–2) is the prerequisite for measuring anything, which is why the project builds it first and then uses it for the audit.

[VERIFY: The team's literature survey names mobility support and key revocation as the planned “delta” extensions, and the manuscript lists completing “the TA-assisted key-revocation extension we proposed earlier” as future work. The repository contains no revocation code. Confirm that the audit replaced the revocation extension as the project's main contribution.]

2.4 Conclusion

The surveyed schemes trade security against overhead in different ways. Blockchain- and certificate-based designs are strong but expensive. Hash/XOR and chaotic-map designs are cheap but have repeatedly been broken. ECC-per-session designs give forward secrecy at a higher computational cost. The base paper's hybrid design is a promising middle ground. However, the comparison that supports it uses unmeasured constants and partly untraceable comparator rows, and it cannot be re-run. The survey of the three comparator papers already shows that their own constants and operation counts differ from those the base paper used. These findings motivate the methodology in Chapter 3: implement the protocol, measure its primitives on real hardware, and recompute the comparison with evidence that can be checked.

CHAPTER: 3

PROPOSED METHODOLOGY

CHAPTER 3

PROPOSED METHODOLOGY

3.1 Introduction

The project proceeds in four stages, shown in Figure 3.1. **Implement:** build a transport-agnostic Python implementation of the base paper's Section III protocol, with demonstration tools, tests and a benchmark harness, following the eight phases of the PRD (cryptographic core; TA and registration; pseudonyms and login gate; mutual-authentication core; security hardening; benchmarking of the proposed scheme; baseline data and report; documentation, CI and release). Then port the device side to C and run it on an ESP8266 against the unmodified Python server. **Measure:** time the primitives the paper's cost model uses on the ESP8266, an AWS Graviton2 core and a laptop, with one C source and checksum-verified outputs. **Audit:** check the paper's tables, equations and figures for consistency, and trace its comparator rows to the original papers. **Recompute:** re-evaluate the paper's cost model with the measured constants, under several readings of the proposed scheme's cost, and use a separately written OMNeT++ model only as a sensitivity analysis. The implementation stage makes the measurements possible, and the measurements and audit address the evidence gap identified in Chapter 2.

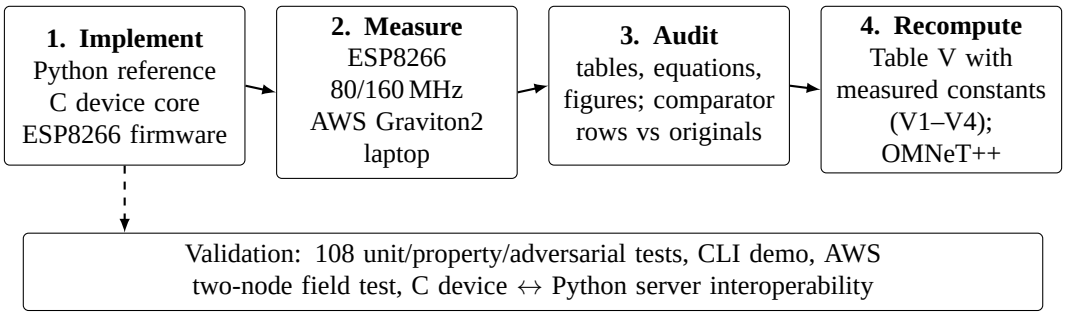


Figure 3.1: Overall project workflow

3.2 Requirement Gathering

Requirements came from two sources. The first was a detailed study of the base paper: its equations (Eq. 8–24), algorithms, threat model, instantiation notes and evaluation tables, together with the comparator schemes reviewed in Chapter 2. The results were written up as a Product Requirements Document (PRD, Draft v0.1, 25 August 2026). The PRD records the decision to implement Section III rather than Section II, defines functional, non-functional, benchmarking and security requirements with iden-

tifiers (FR, NFR, BR, SEC) that the code and tests reference, and sets acceptance criteria. The second source was the project review cycle. The audit notes record a mentor's condition that the paper's Fig. 1 be reproduced (“their benchmark/dataset, or a model resolving to the same numbers”) before measured hardware results are compared with it; this led to the reproduction attempt and the replication study in Chapter 4. **[VERIFY: The PRD says the Section III decision was “confirmed with stakeholders”, and the audit notes mention a “mentor”. Add the actual review meetings, guide inputs or other requirement-gathering techniques used.]**

3.2.1 Functional Requirements

Table 3.1: Functional requirements (from the PRD) and implementation status

ID	Requirement	Status
FR-1	Primitives: NIST P-256; SHA-256 for $H_{1,2,3}$; HKDF-SHA256; AES-256-GCM (tag as MAC); CSPRNG nonces of 128 bits (192/256 configurable)	Implemented (`crypto/`)
FR-2	Offline TA: $\lambda_X = H_1(ID_X \| s)$ mod n; optional $P_X = \lambda_X G$; no network I/O; batch enrolment from a manifest	Implemented (`ta.py`); no-socket invariant tested
FR-3	Pairwise root $k^{\star}_{ij} = \mathrm{KDF}(\lambda_{V_i} \| \lambda_{R_j})$ on both sides, with caching and an invalidation hook	Implemented (`entity.py`)
FR-4	Pseudonym $PID_{V_i,t} = H_2(\lambda_{V_i} \| t)$; server resolves it with constant-time comparison	Implemented (`device.py`, `server.py`)
FR-5	Two-message exchange (Eq. 12–19); typed errors; 5 s session timeout; three-attempt block-list with broadcast hook	Implemented
FR-6	Optional login pre-check $H_D = h(MAC_D \| PWD)$	Implemented, disabled by default
FR-7	Pluggable coverage predicate and $Allow_{ij}(t)$ gate	Implemented (default: always in range)
FR-8	Stub interface for TA-assisted rekey (Eq. 24)	Stub only (raises `NotImplemented-Error`)

3.2.2 Non Functional Requirements

Table 3.2: Non-functional requirements (from the PRD) and observed status

Category	Requirement	Observed in this project
Correctness	100% of state-machine branches covered; Eq. 20 holds in $\geq$10,000 randomised trials	10,000/10,000 trials passed; line coverage 97% (not 100% branch coverage; see Table 4.3)
Performance	Two-message authentication $<$5 ms median, excluding network	$\approx$0.10 ms median (device 0.039 + server 0.061 ms)
Portability	Python $\geq$3.11; Linux/macOS/Windows; single `pip install -e .`	CI on Ubuntu with Python 3.11 and 3.12; local run on Linux with 3.14.8. macOS/Windows not verified
Reproducibility	Pinned `requirements-lock.txt`; report records CPU model, Python and library versions	Lock file present; report records Python, platform and library versions but only "x86_64" for the CPU
Test coverage	$\geq$90% on `protocol/` and `crypto/`, enforced in CI	97%; CI uses `--cov-fail-under=90`
Logging	Structured JSON logs per attempt, no secrets	Server logs JSON events (Listing 4.3); device CLI prints plain text
Licensing	Permissive licence; citation in README and NOTICE	Apache-2.0; citation present in both

3.2.3 User Requirements

Table 3.3: User requirements (from the PRD)

User	Need	How the system serves it
Security reviewer / auditor	Verify the claimed properties against a concrete artefact	Unit tests per security goal and a documented adversarial suite
Research / applied-crypto engineer	Reproduce and sanity-check the paper's performance claims	Benchmark harness producing comparable numbers and charts
Product engineering	Decide whether to invest in a hardened version	Working reference implementation plus a list of non-production gaps
Future firmware team	Reuse validated protocol logic on embedded targets	Protocol state machines and message formats independent of the demo transport

3.3 Proposed Algorithms

Selection and justification. The protocol follows Section III of the base paper. Section II was rejected because it reuses H_D/R_D across sessions with no fresh nonce, which makes it replayable, and because the paper's security sketch covers only Section III. The concrete primitives follow the paper's instantiation notes: SHA-256 for H_1, H_2, H_3; HKDF-SHA256 [24] as the KDF; AES-256-GCM [25] for Enc/Dec, with the GCM tag serving as the MAC (μ_1, σ_2); NIST P-256 as the curve; and a security parameter $\ell = 128$ bits for nonces.

Offline registration (TA). The TA samples a master secret s and, for each entity X, computes

$$\lambda_X = H_1(ID_X \,\|\, s) \bmod n, \tag{3.1}$$

optionally exports $P_X = \lambda_X G$, and gives each entity the λ values of its trusted peers. No network I/O is performed.

Pairwise root, pseudonym and session key.

$$k^{\star}_{ij} = \text{KDF}(\lambda_{V_i} \,\|\, \lambda_{R_j}), \tag{3.2}$$

$$PID_{V_i,t} = H_2(\lambda_{V_i} \,\|\, t), \tag{3.3}$$

$$K_{ij,t} = \text{KDF}(k^{\star}_{ij} \,\|\, N_i \,\|\, N_j \,\|\, t). \tag{3.4}$$

Distinct HKDF `info` strings separate the pairwise-root and session-key derivations.

Messages. The device sends $M_1 = \langle PID_{V_i,t}, C_1, \mu_1 \rangle$, where (C_1, μ_1) is a single AES-GCM encryption, under $k^{\star}_{ij}$, of the plaintext $PID_{V_i,t} \,\|\, N_i \,\|\, t$. The server replies with $M_2 = \langle ID_{R_j}, C_2, \sigma_2 \rangle$, where (C_2, σ_2) encrypts $N_j \| t$ with $PID_{V_i,t} \| N_i$ bound as additional authenticated data. This reproduces the coverage of $\sigma_2 = \text{MAC}(PID \| N_i \| N_j \| t)$ in the paper's Eq. 17 while sending only $N_j \| t$ as ciphertext. Correctness requires

$$\Pr\left[K^{(V)}_{ij,t} = K^{(R)}_{ij,t}\right] = 1, \tag{3.5}$$

and access is granted only if $Allow_{ij}(t) = cov_{ij}(t) \wedge Auth_{ij}(t)$.

Epoch handling (implementation decision). The paper does not specify how t evolves or how clock drift is handled. In this implementation, the epoch is tracked per peer relationship and incremented after each request. The server searches for a matching pseudonym only in the window $[\, last - 16,\ last + 8 \,]$ around the last epoch it accepted from that peer, and rejects any epoch $\leq last$ as a replay.

```
Input: peer id R_j
if R_j is block-listed: raise BlockedEntity
if cov_ij(t) = False: raise CoverageError
```

```
k* ← cached or KDF(λ_{V_i} ‖ λ_{R_j})
t ← epoch[R_j]; PID ← H_2(λ_{V_i} ‖ t); N_i ← CSPRNG(128 bits)
(iv, C_1, μ_1) ← AES-GCM-Enc_{k*}(PID ‖ N_i ‖ t)
store pending session (PID, N_i, t, k*, now)
epoch[R_j] ← t + 1
return M_1 = (PID, iv, C_1, μ_1)
```

Listing 3.1: Algorithm: device builds M_1 (`Device.build_auth_request`)

```
for each known peer V, for each t' in window(V):
    if H_2(λ_V ‖ t') = PID (constant-time):
        if t' ≤ last_accepted[V]: raise ReplayDetected
        (peer, t) ← (V, t'); break
if no match: raise UnknownPeerError
if peer block-listed: raise BlockedEntity       // before any
   decryption
if cov_{ij}(t) = False: record failure; raise CoverageError
k* ← cached or KDF(λ_V ‖ λ_{R_j})
plaintext ← AES-GCM-Dec_{k*}(C_1, μ_1) // failure: record failure,
   IntegrityError
check plaintext.PID = PID and plaintext.t = t // else: record failure
   , IntegrityError
last_accepted[peer] ← t; reset failure count
N_j ← CSPRNG(128 bits); K_{ij,t} ← KDF(k* ‖ N_i ‖ N_j ‖ t)
(iv, C_2, σ_2) ← AES-GCM-Enc_{k*}(N_j ‖ t, AAD = PID ‖ N_i)
return M_2 = (ID_{R_j}, iv, C_2, σ_2), K_{ij,t}
```

Listing 3.2: Algorithm: server verifies M_1 and builds M_2 (`Server.handle_auth_request`)

```
pending ← pop pending session for ID_{R_j} // none: UnknownPeerError
if elapsed > 5 s: record failure; raise SessionTimeoutError
(N_j ‖ t') ← AES-GCM-Dec_{k*}(C_2, σ_2, AAD = PID ‖ N_i) // failure:
   IntegrityError
if t' ≠ t: record failure; raise IntegrityError
reset failure count
return K_{ij,t} = KDF(k* ‖ N_i ‖ N_j ‖ t)
```

Listing 3.3: Algorithm: device verifies M_2 (`Device.complete_auth`)

A failure counter per peer is incremented on every verification failure. On the third consecutive failure the peer is block-listed and listeners are notified, which implements the paper's "maximum three attempts" rule and its broadcast of the blocked identifier.

Measurement and cost-model recomputation. The base paper costs each scheme per role as

$$\text{cost} = n_H T_H + n_{SE/D} T_{SE/D} + n_{ECM} T_{ECM} + n_{fe} T_{fe} + n_C T_C, \tag{3.6}$$

with operation counts n from its Table V and assumed times T from its Table III. This project keeps the paper's counts and replaces the times with measured medians. A single C benchmark built from the same source on every platform times SHA-1 and SHA-256, HMAC, HKDF-SHA256, AES-256-GCM (with the key schedule paid or cached, and three BearSSL AES implementations: constant-time `aes_ct`, `aes_small` and table-based `aes_big`), bare AES and AES-CTR, and P-256 and X25519 scalar multiplication. Each value is the median of 100 runs (24 for elliptic-curve operations), and every output is checksummed so that all platforms can be shown to compute the same results.

Table V's count for the proposed scheme, $2T_H + 2T_{SE/D}$ per side, omits the session-key derivation that the paper's own Eq. 16 and Eq. 22 require. The proposed scheme is therefore evaluated under four readings:

- **V1:** Table V as printed;
- **V2:** Table V plus one KDF per side;
- **V3:** Eq. 22 with an AEAD (device: one KDF and one AEAD operation; server: one KDF and two AEAD operations);
- **V4:** Table V's "$2T_H$" read as the pseudonym hash plus the KDF, the reading most favourable to the paper.

The definition of $T_{SE/D}$ (full GCM, GCM with cached key, CTR without MAC, one AES block) and of T_H (one or two SHA-1 blocks) is also swept. The key quantity is $r = T_{SE/D}/T_H$: the proposed scheme beats a hash-only scheme only if r is below about 2.0–4.5, depending on the cost column.

Consistency audit. Tables III–VI were transcribed from 300-DPI renders of the paper and re-read at 600 DPI. Tables III, IV and VI were checked by optical character recognition, and Table V by two manual cell-by-cell readings. Bar heights in Figs. 1–3 were read from 600-DPI renders. For each table and figure, the proposed scheme's reduction was computed under nine definitions (against the best, mean, median and worst competitor, averaged over the x-axis, over all scheme/x pairs, at the first and last point, and as a ratio of sums) to test whether any reproduces 17%, 21% or 12%.

3.4 Gantt chart for Timeline

Figure 3.2 is built from the repository's commit history (25 August to 27 October 2026). Each bar spans the first and last commit date of an activity. Commit dates record when work was *committed*, not when it started, so bars of one day usually stand for

work done in the days before. **[VERIFY: Commit dates are the only dated evidence in the repository. Replace or extend the bars with the actual planned and completed dates, including work before 25 August (topic selection, base paper selection) and any review dates.]**

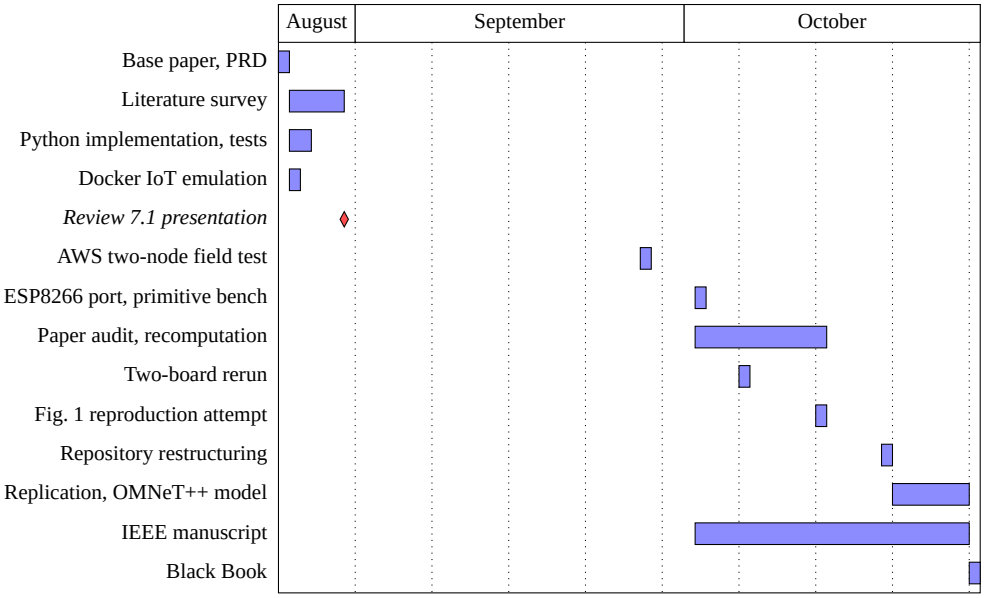


Figure 3.2: Gantt chart of project activities (from repository commit dates)

3.5 Architectural Diagram

Figure 3.3 shows the layered architecture of the `hybrid_ecc_auth` package. The protocol core is transport-agnostic: it consumes and produces byte strings, and only the `demo/` layer knows about TCP. Dependencies point downwards only. `protocol/` uses `crypto/` and `storage/`, while `demo/` and `bench/` use `protocol/`. Outside the Python package, `firmware/esp8266/` re-implements the device side of `protocol/` in C on BearSSL and speaks the same wire format, and `research/` holds the audit and replication tooling, which reads the package and the measured results but is not imported by it.

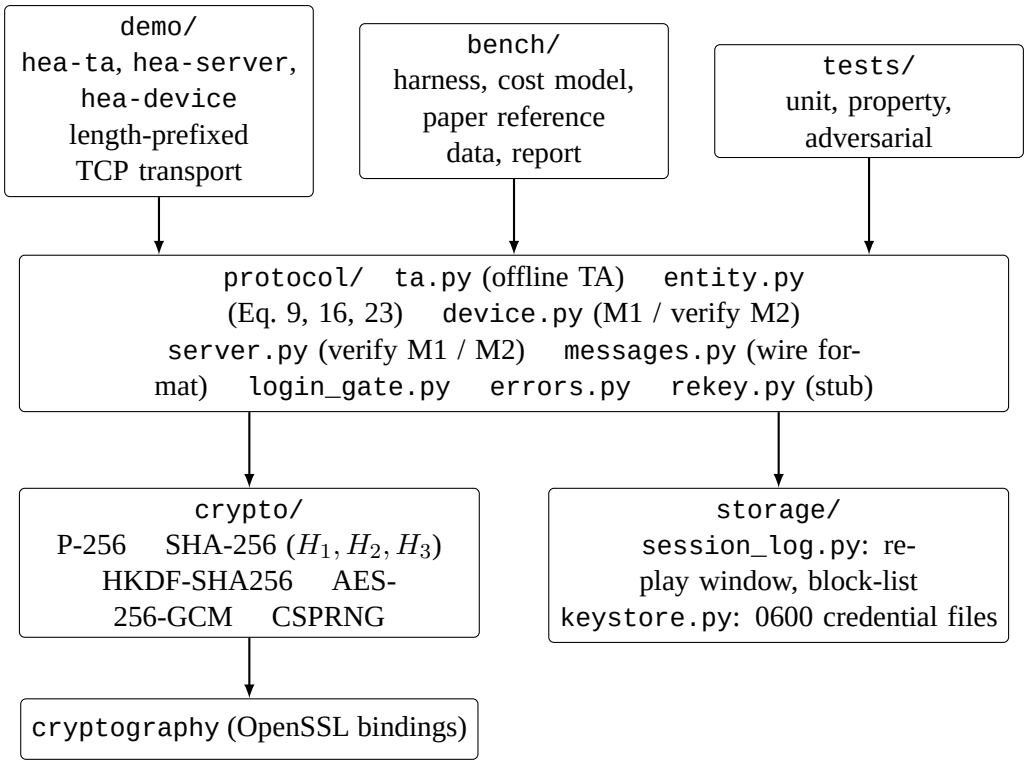


Figure 3.3: Layered architecture of the implementation

3.5.1 UML Diagrams

Use-case diagram. Figure 3.4 shows the actors who interact with the system through the command-line tools and the developer workflow described in the README.

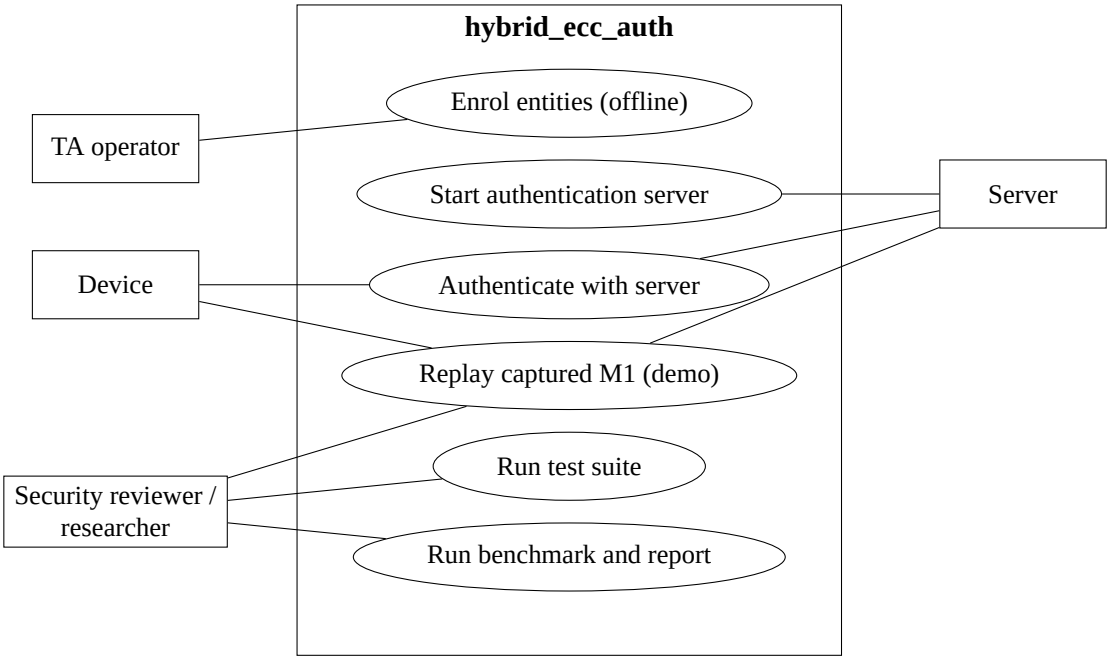


Figure 3.4: Use-case diagram

Class diagram. Figure 3.5 shows the main classes. `Device` and `Server`

share pairwise-root caching, coverage checking and session-key derivation through the `Entity` base class. Each owns a `SessionLog` for replay and block-list state.

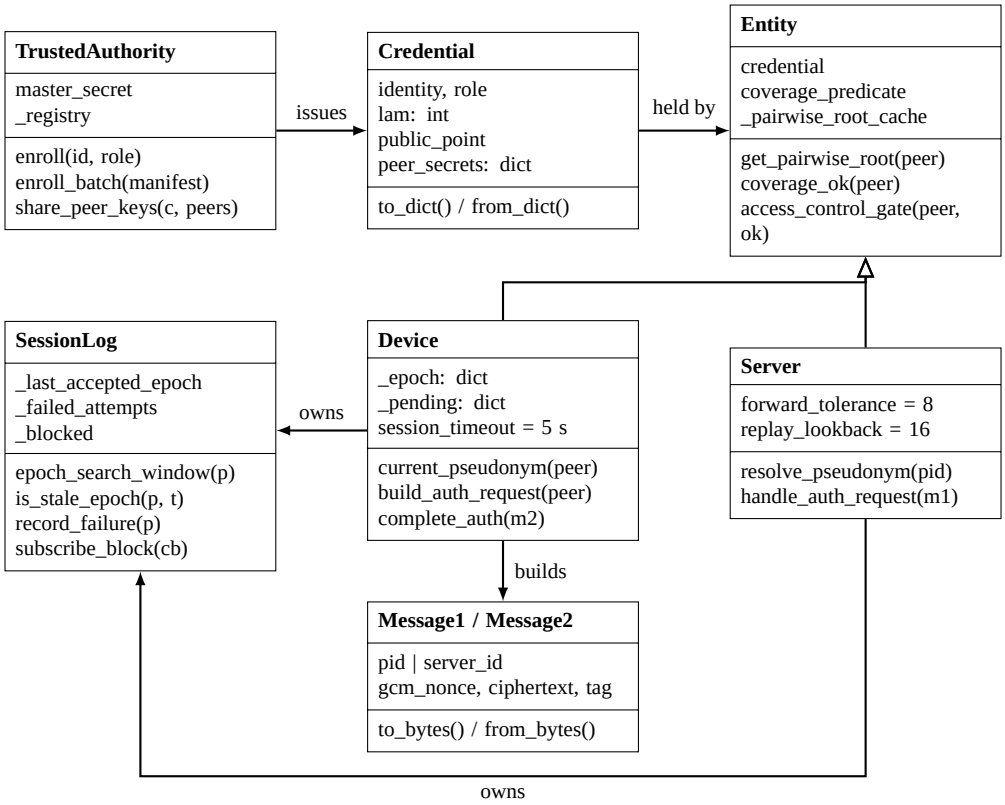


Figure 3.5: Class diagram of the protocol core

Sequence diagram. Figure 3.6 shows the complete flow: offline provisioning followed by the two-message online exchange.

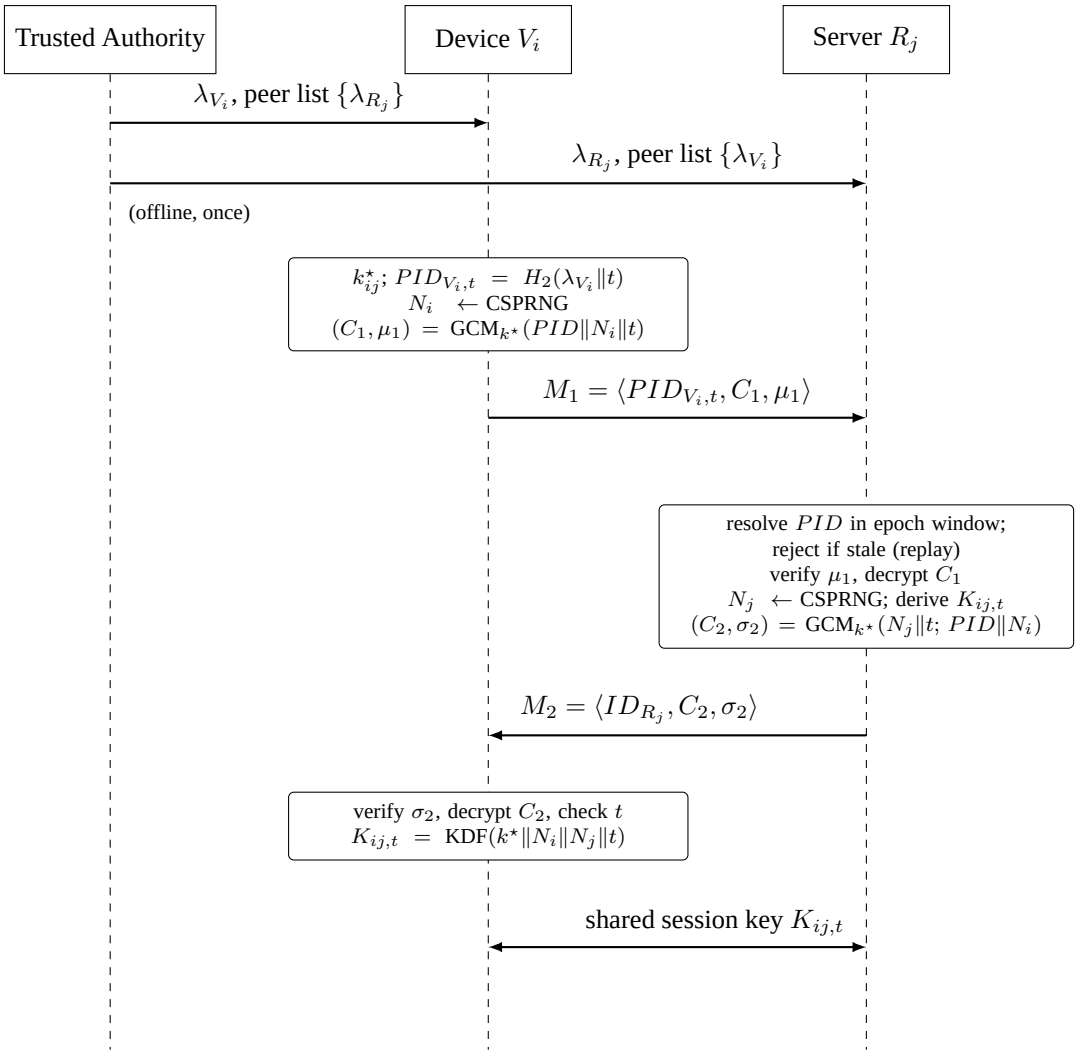


Figure 3.6: Sequence diagram: offline registration and two-message mutual authentication

3.5.2 Block Diagram

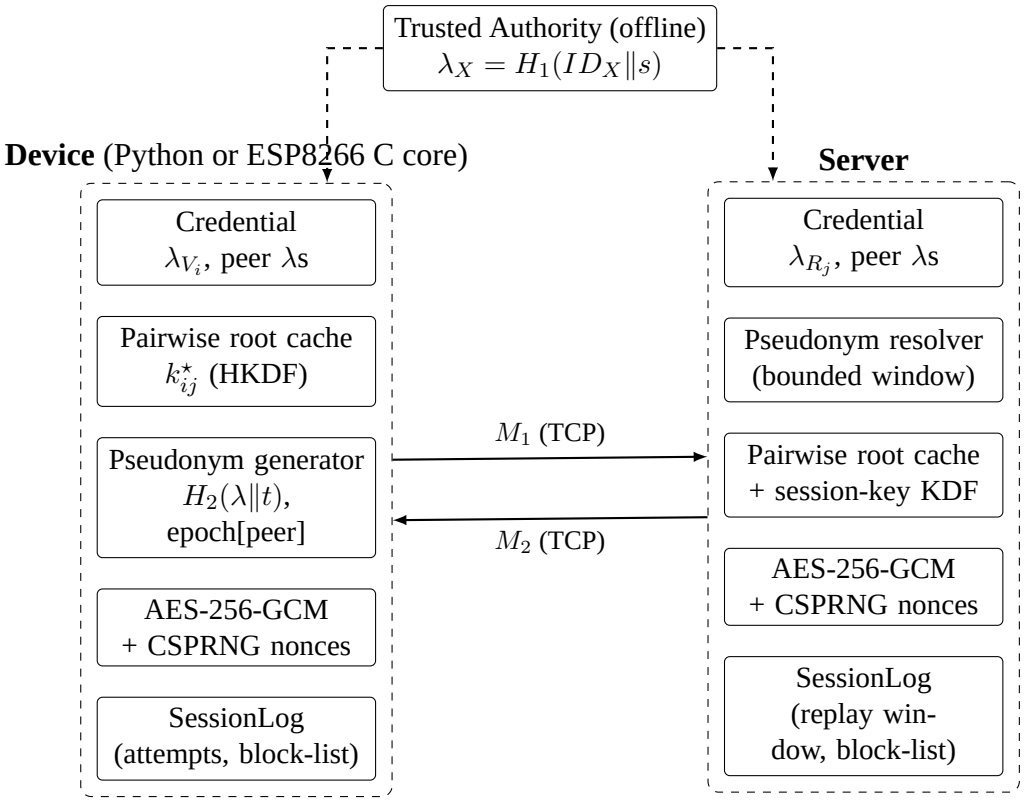


Figure 3.7: Block diagram of the system components

3.5.3 Data Flow Diagram

Level 0 (context). Figure 3.8 treats the authentication system as a single process. The manifest enters the TA, and credential bundles are written for each entity. Devices send authentication requests and receive responses, and the server writes JSON log events.

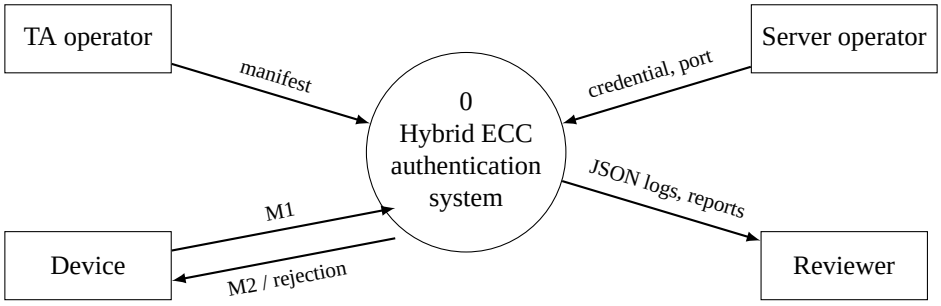


Figure 3.8: Data flow diagram, level 0

Level 1. Figure 3.9 breaks the system into its main processes and the data stores they read and write.

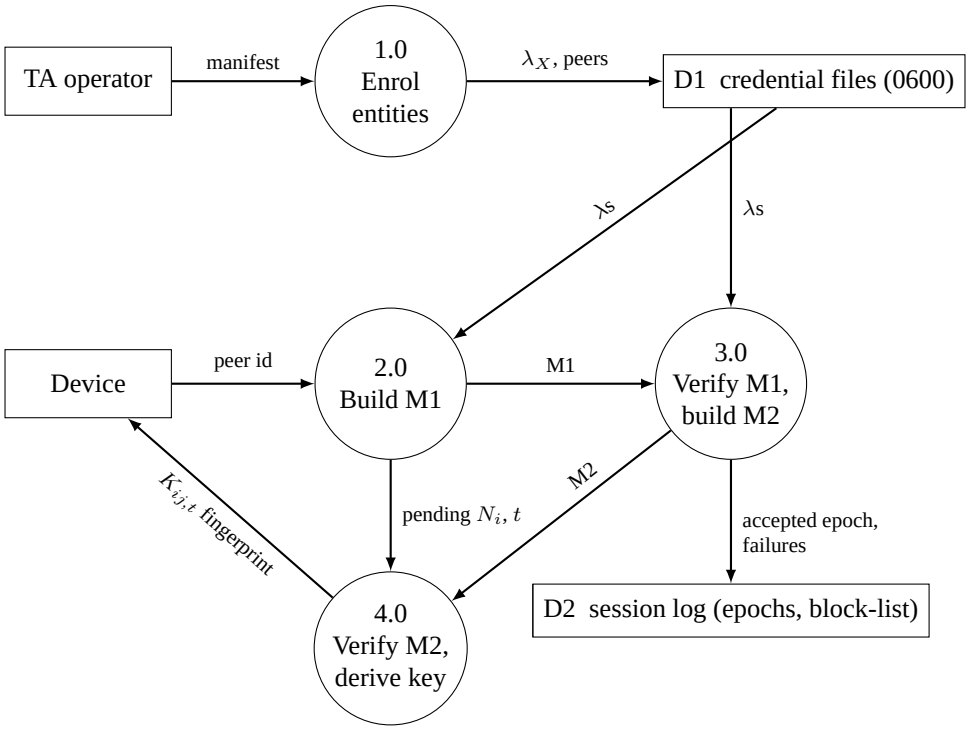


Figure 3.9: Data flow diagram, level 1

3.5.4 Timeline Chart

Where the Gantt chart shows activities, Figure 3.10 shows the project's milestones: dated deliverables and decision points recorded in the repository.

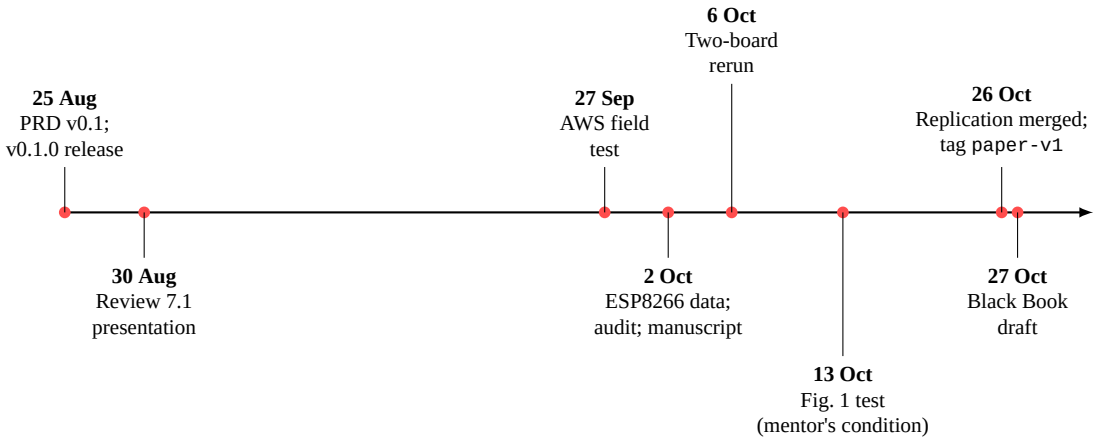


Figure 3.10: Project milestone timeline (2026)

3.6 Hardware Requirements

Table 3.4 lists the hardware on which the project's results were produced, as recorded in the repository's result files and reports.

Table 3.4: Hardware used for development and measurement

Component	Specification (as recorded)	Used for	Availability
NodeMCU ESP8266 boards (2)	Tensilica L106, 80 MHz (also run at 160 MHz), about 80 KB RAM, no cryptographic hardware; chip IDs 00d37d28 and 00d377aa	Device firmware, primitive benchmark	**[MISSING:]**
Apple-silicon laptop	Model not recorded	Primitive benchmark (BearSSL and OpenSSL), server scaling	**[MISSING:]**
Windows 11 laptop	16 logical CPUs; WSL2 Ubuntu 22.04.5 with 7 GB RAM	Replication study, OMNeT++	**[MISSING:]**
Linux laptop	Intel Core i5-12500H, 16 logical CPUs, 15 GiB RAM, kernel 6.18.57	Re-runs for this report	**[MISSING:]**
AWS t4g.nano (2 + 1)	Graviton2 (Neoverse-N1), 2 vCPU, 512 MiB, Amazon Linux 2023, us-east-1	Two-node field test; primitive benchmark	Paid, on demand
GitHub-hosted runners	`ubuntu-latest`	CI	Free**[VERIFY:]**

No 8- or 16-bit microcontroller, MCU with cryptographic hardware, or energy-measurement equipment was used. **[MISSING: Whether each laptop and the ESP8266 boards are personally owned or institute-provided, and the minimum specification, if the department requires one.]**

3.7 Software Requirements

Table 3.5: Software requirements (versions from `pyproject.toml`, lock files and result logs)

Item	Purpose	Version
Python	Reference implementation, audit and replication scripts	≥3.11 (3.11.9, 3.11.16, 3.14.8 used)
`cryptography` (OpenSSL)	P-256, HKDF-SHA256, AES-256-GCM in Python	50.0.0 / 50.0.1
pytest, pytest-cov, hypothesis	Unit, property and coverage testing	9.1.1, 7.1.0, 6.165.10
matplotlib, numpy	Charts and analysis	3.11.x, 2.4–2.5
C compiler, Make	Native build of the C device core and benchmark	–
BearSSL (ESP8266 fork)	AES-GCM, SHA, HKDF, P-256 in C	as shipped with the ESP8266 Arduino core
PlatformIO, Arduino core for ESP8266	Firmware build and flashing (`espressif8266`, `nodemcuv2`)	–
OpenSSL libcrypto	Hardware-accelerated primitive comparison on the laptop	–
OMNeT++	Network model of the paper's Table II setting	6.0.3 (Academic Public License)
SimPy, PyMuPDF, PyYAML	Cross-check simulation; PDF rendering; data extraction	4.1.2, 2.0, 6.0.3
Docker, Docker Compose	Multi-node IoT emulation and benchmark runner	–
AWS CLI, Amazon Linux 2023	Cloud field test	AL2023.12
Git, GitHub Actions	Version control and CI (Python 3.11, 3.12)	–
LaTeX	Manuscript and report	–

3.8 Cost Estimation

Table 3.6: Cost estimation

Sr. No.	Item / Resource	Description	Estimated Cost (Rs.)
1	Hardware	Laptops (Table 3.4); 2 NodeMCU ESP8266 boards	**[MISSING: laptops existing? board price]**
2	Software / Tools	All tools in Table 3.5 are free or open source; project released under Apache-2.0	0 (free/open-source)
3	Cloud / Hosting	AWS t4g.nano at US$0.0042/h each (2 nodes for the field test plus 1 for benchmarking) and 2 × 8 GB gp3 volumes; instances terminated after use. Public GitHub repository and Actions	**[MISSING: actual AWS bill; GitHub 0?]**
4	APIs / Services	None	0
5	Other Expenses	Base paper via the institute's IEEE Xplore subscription (existing resource)	**[MISSING: printing, binding, publication fees]**
	Total Estimated Cost		**[MISSING:]**

All software is free or open source, and no paid API is used. The only recurring cost was the AWS instances, which the field-test report prices at US$0.0042 per hour each, plus storage. Since the instances ran only for the duration of the tests, the bill depends on how long they ran, which is not recorded. The ESP8266 boards are the only hardware bought specifically for the project, if they were bought.

3.9 Conclusion

The methodology has four stages: implement the base paper's Section III protocol in Python and in C for an ESP8266; measure the primitives of the paper's cost model on three hardware classes with one benchmark source; audit the paper's tables, equations and figures; and recompute its comparison under four readings of the proposed scheme's cost, with an OMNeT++ model used only for sensitivity analysis. Requirements come from the PRD and the review cycle, and are expressed as eight functional and seven non-functional requirements. A documented epoch window fills a gap in the paper's specification. The layered architecture separates primitives, protocol, storage, demonstration and benchmarking, so the protocol core can be ported, as the ESP8266 firmware

shows. Hardware and software needs are modest and mostly free, with small cloud and board costs. This methodology produces the implementation and evidence presented in Chapter 4.

CHAPTER: 4

IMPLEMENTATION & RESULTS

CHAPTER 4

IMPLEMENTATION & RESULTS

4.1 Introduction

This chapter describes what was built and what it produced. The implementation has five parts: the Python reference implementation; the C device core and ESP8266 firmware; a two-node cloud deployment; the primitive benchmark and paper-audit scripts; and the replication pipeline with its OMNeT++ model. Five aspects are evaluated: functional correctness, security behaviour, end-to-end operation on three kinds of platform, the measured cost of the primitives and of the protocol, and the base paper's claims recomputed with that evidence. Results marked *re-run* were regenerated for this report on the Linux laptop in Table 3.4; their raw outputs are in `BlackBook_Draft/run_outputs/`. All other results are the team's committed measurements, with their source file named.

4.2 Implementation

Code base. Table 4.1 summarises the repository.

Table 4.1: Implementation components (lines include comments)

Component	Files	Lines	Contents
`hybrid_ecc_auth/crypto/`	5	181	P-256, $H_1/H_2/H_3$, HKDF, AES-GCM, nonces
`hybrid_ecc_auth/protocol/`	8	706	TA, Entity, Device, Server, messages, login gate, errors, rekey stub
`hybrid_ecc_auth/storage/`	2	154	Replay window, block-list, keystore
`hybrid_ecc_auth/demo/`	4	432	TCP framing, `hea-ta`, `hea-server`, `hea-device`
`hybrid_ecc_auth/bench/`	4	968	Cost model, paper reference data, harness, report
`hybrid_ecc_auth/tests/`	15	1,385	Unit, property, adversarial
`firmware/esp8266/` (core, firmware)	3	474	Portable C protocol core, Arduino firmware
`firmware/esp8266/` (bench, host, tools)	7	534	Primitive benchmark, native build, interop tools
`research/paper_audit/`	12	656	Audit, recomputation, protocol checks
`research/reference_replication/`	27	3,944	Replication scripts, OMNeT++ model, tests

Python reference implementation. Each cryptographic primitive is a thin wrapper over the `cryptography` library. λ_X is SHA-256 of identity and master secret reduced modulo the P-256 order. AES-GCM uses a fresh 96-bit IV per message and returns the 16-byte tag separately so that it can serve as μ_1 or σ_2. Listing 4.1 shows the core of M_1 construction. The plaintext layout, PID (32 B) $\|$ N_i (16 B) $\|$ t (8 B), is an implementation choice, because the paper does not specify byte layouts.

```
k_star = self.get_pairwise_root(peer_id)
epoch = self._epoch.get(peer_id, 0)
pid = self.current_pseudonym(peer_id, epoch)
n_i = generate_nonce(self.nonce_bits)

plaintext = pid + n_i + epoch.to_bytes(8, "big")
gcm_nonce, ciphertext, tag = aead.encrypt(k_star, plaintext)
msg1 = Message1(pid=pid, gcm_nonce=gcm_nonce, ciphertext=ciphertext,
    tag=tag)
```

Listing 4.1: Building M_1 (excerpt from `protocol/device.py`)

On the server, the block-list check runs before any decryption, and the response binds $PID\|N_i$ as additional authenticated data (Listing 4.2).

```
self.session_log.accept_epoch(peer_identity, epoch)
self.session_log.record_success(peer_identity)

n_j = generate_nonce(self.nonce_bits)
session_key = derive_session_key(k_star, n_i, n_j, epoch)

aad = msg1.pid + n_i
plaintext2 = n_j + epoch.to_bytes(8, "big")
gcm_nonce2, ciphertext2, tag2 = aead.encrypt(k_star, plaintext2, aad=
    aad)
```

Listing 4.2: Building M_2 (excerpt from `protocol/server.py`)

Messages are compact JSON with base64-encoded fields and a version number. The demo transport adds a 4-byte length prefix and a success/error envelope. `hea-ta enroll` reads a manifest, enrols every entity, cross-issues peer keys and writes credential files with mode 0600. `hea-server` serves requests and logs one JSON event per attempt. `hea-device` prints a fingerprint of the session key, never the key itself, and `--replay-last` resends the previous M_1. The benchmark harness measures message sizes, processing time (1,000 warm and 100 cold trials) and persisted-state size, and sweeps server cost from 100 to 1,800 enrolled devices. GitHub Actions runs the tests with a 90% coverage gate on Python 3.11 and 3.12. A Docker Compose setup runs a multi-node emulation and the benchmark.

C device core and ESP8266 firmware. `firmware/esp8266/lib/hea_proto` is a portable C port of the device side on BearSSL [7] (AES-256-GCM, SHA-256, HKDF-SHA256), with no Arduino dependencies. It produces the identical framed-JSON wire format, so it talks to the unmodified Python server. `src/main.cpp` adds Wi-Fi, a handshake loop, per-step timings and a replay test. Unlike the Python reference, the firmware persists the epoch to flash. Offline enrolment stays on the laptop, and the board receives only λ_{device} and λ_{server}. The same core builds natively, and `tools/interop.py` checks it against the Python server without hardware.

Primitive benchmark. `firmware/esp8266/bench/prim_bench.c` is the single C source used on every platform. On the ESP8266 it runs with Wi-Fi off and cycle-counter timing; P-256 runs on a temporary heap stack because the 4 KB task stack is too small. `research/paper_audit/libcrypto_bench.c` times OpenSSL [26] with hardware AES and SHA on the laptop, per message, with a low-level HKDF verified equal to OpenSSL's.

Audit and recomputation scripts. `research/paper_audit/` contains the table transcription with OCR verification (`paper_tables.py`, `verify_tables.py`), the consistency audit (`audit.py`), figure digitisation (`figures_audit.py`), the re-

computation with measured constants (`reeval.py`, `eq22_variants.py`), the Fig. 1 test (`fig1_repro.py`), and the forward-secrecy and server-scaling checks.

Replication pipeline. `research/reference_replication/` reproduces the three comparator papers' own numbers, audits how the base paper used them, compares all schemes under three sets of constants (the base paper's, the comparator papers' own, and the measured ones), and runs a network simulation of the base paper's Table II setting. The simulation is an OMNeT++ 6.0.3 [27] model written by the team, because the base paper publishes no simulation files, cross-checked against an independent SimPy model. The pipeline reports 46 passing tests, and its OMNeT++ sweep (3,385 runs) is byte-identical when repeated. **[VERIFY: The replication suite needs PyYAML, SimPy and PyMuPDF, which are not installed in this environment, so its tests were not re-run for this report.]**

Cloud deployment. On 27 September 2026 the Python implementation was deployed on two AWS t4g.nano instances (device node and server node) in one availability zone. The repository's own scripts ran there: the test suite, CLI walkthrough, handshake loops, security scenarios over the network, and the full benchmark.

Traceability. Table 4.2 maps the paper's symbols to code.

Table 4.2: Paper symbol to code traceability

Paper symbol	Eq.	Code reference
λ_X	8	`protocol/ta.py::derive_long_term_secret`
$k^{\star}_{ij}$	9	`protocol/entity.py::derive_pairwise_root`
$cov_{ij}(t)$	10	`protocol/entity.py::Entity.coverage_ok`
$PID_{V_i,t}$	11	`protocol/device.py::` `Device.current_pseudonym`
N_i, C_1, μ_1, M_1	12–15	`protocol/device.py::` `Device.build_auth_request`
$K_{ij,t}$	16	`protocol/entity.py::derive_session_key`
σ_2, C_2, M_2	17–19	`protocol/server.py::` `Server.handle_auth_request`
Correctness	20	`tests/property/test_session_key_agreement.py`
$Allow_{ij}(t)$	23	`protocol/entity.py::` `Entity.access_control_gate`
Rekey	24	`protocol/rekey.py` (stub)

4.3 Results of Implementation

Functional and adversarial tests (re-run)

After pulling the latest repository, the test suite ran 108 tests: 97 unit, 2 property-based and 9 adversarial (SEC-03 runs twice). All passed. The property test ran 10,000 randomised sessions with zero key mismatches. Line coverage of `protocol/` and `crypto/` is 97% (Table 4.3). The same suite also passed on the AWS device node (108 passed in 10.8 s, 97% coverage).

Table 4.3: Line coverage of `protocol/` and `crypto/`

Module	Statements	Missed	Coverage
`crypto/` (5 modules)	67	1	99%
`protocol/device.py`	71	4	94%
`protocol/server.py`	62	3	95%
`protocol/messages.py`	54	3	94%
`protocol/ta.py`	47	1	98%
other `protocol/` modules	65	0	100%
Total	**366**	**12**	**97%**

Table 4.4: Adversarial test cases (PRD Section 7.2), all passing

ID	Scenario	Expected and observed result
SEC-01	Replay a captured valid M_1	Rejected: stale epoch
SEC-02	Replay a captured M_2 into another session	Rejected: σ_2/nonce binding
SEC-03	Flip one bit in C_1 or μ_1	Rejected: tag verification (both cases)
SEC-04	Forge M_1 for a known PID without λ_{V_i}	Rejected: invalid μ_1
SEC-05	Three consecutive failures	Block-listed; 4th attempt rejected before any cryptography
SEC-06	Compare pseudonyms across epochs	Indistinguishability smoke test passes
SEC-07	Two independent honest runs	Different session keys
SEC-08	Out-of-coverage device	Rejected at $Allow_{ij}(t)$

End-to-end demonstration on one machine (re-run)

```
$ hea-ta enroll --manifest manifest.json --out-dir creds/
Enrolled 'device-001' (role=device) -> creds/device-001.json
Enrolled 'server-A' (role=server) -> creds/server-A.json
2 entities enrolled and cross-issued peer keys. Offline phase complete -- no network I/O
    occurred.
```

```
$ hea-device authenticate --credential creds/device-001.json --server-id server-A --server-addr
    127.0.0.1:18443
Mutual authentication succeeded with 'server-A'.
  session key fingerprint: e1507ce5d2d12bcf (SHA-256[:16] of K_ij,t -- never the raw key)
  round-trip latency: 4.218 ms

$ hea-device authenticate ... --replay-last
[replay-demo] resending previously captured M1 (216 bytes) to 127.0.0.1:18443 ...
REJECTED by server: ReplayDetected: pseudonym for 'device-001' reuses stale epoch 0 (round-trip
    2.286 ms)

server log:
{"event": "auth_attempt", "outcome": "accepted", "peer_identity": "device-001", "latency_ms":
    1.5397, ...}
{"event": "auth_attempt", "outcome": "rejected", "error_type": "ReplayDetected", ..., "
    latency_ms": 0.0884, ...}
```

Listing 4.3: CLI demo output: enrolment, successful authentication, replay rejection and server log

ESP8266 device against the Python server

The C device core completed 12 of 12 handshakes natively and 20 of 20 on a NodeMCU ESP8266 over Wi-Fi against the unmodified Python server. In every case the session-key fingerprints on both sides were identical, and a replayed M_1 was rejected. On the ESP8266 at 80 MHz, building M_1 took 1.67 ms and verifying M_2 and deriving the key took 3.59 ms: 5.26 ms of device computation per handshake (mean of 20), and 52 ms end to end over Wi-Fi. M_1 is 216 bytes. **[VERIFY: These figures are as reported in the team's manuscript; the raw handshake log is not committed, and the native interop check was not re-run because building it requires downloading BearSSL.]**

Two-node cloud field test

Table 4.5: AWS t4g.nano field test (device and server on separate instances; 2,000 handshakes, 0 failures, 2,000 unique session keys)

Segment (ms)	**p50**	**p95**	**p99**
Device: build M_1	0.041	0.057	0.070
Network round trip incl. server	0.617	0.804	0.917
Device: verify M_2 + KDF	0.067	0.100	0.131
End-to-end handshake	0.728	0.931	1.059
Server `handle_auth_request` (server log)	0.132	0.163	0.194

About two-thirds of a handshake was TCP round trip. Sequential throughput was 1,326 handshakes/s. Messages were 216 B (M_1) and 136 B (M_2), 409 B of TCP

payload with framing. The device process peaked at 30 MB of memory. Source: `docs/experiments/aws_demo_2026-09-27/`.

Protocol cost on a laptop (re-run)

Table 4.6: Measured message sizes per session

Message	Raw fields (bytes)	Raw total	JSON wire
M_1	PID 32, IV 12, ciphertext 56, tag 16	116	216
M_2	server ID 14, IV 12, ciphertext 24, tag 16	66	142
Session		**182 (1,456 bits)**	**358**

The M_2 size depends on the server identifier: 14 characters here, 8 (`server-A`) in the AWS test.

Table 4.7: Measured processing time per session on the Linux laptop (ms)

Operation	Trials	Mean	Median	p95
Device total (warm)	1000	0.047	0.039	0.090
Server: handle M_1 (warm)	1000	0.073	0.061	0.143
Device total (cold)	100	0.047	0.047	0.053
Server total (cold)	100	0.067	0.068	0.072

Persisted state is 424 bytes for a device and 449 bytes for a server with one peer each. Figure 4.1 compares the measured message size with the paper's reported values for the other schemes.

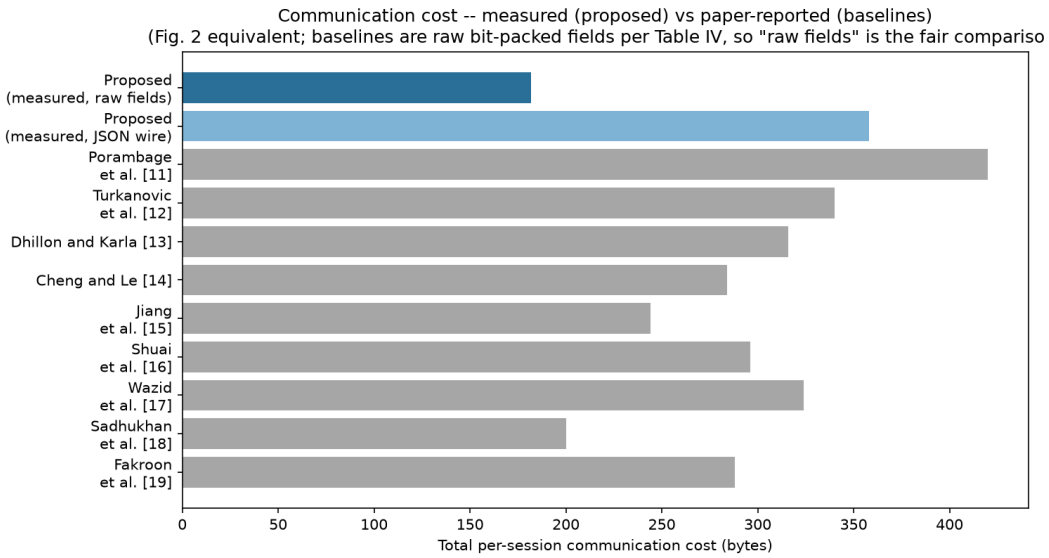


Figure 4.1: Communication cost: measured (proposed) vs. paper-reported baselines (Table IV)

Measured primitive costs

Table 4.8: Paper constants vs. measured medians (GCM: 56-byte AES-256-GCM including key setup, `aes_big`/`aes_ct`)

	Paper (Table III)	**ESP8266 80 MHz**	**ESP8266 160 MHz**	**Graviton2 t4g.nano**
T_H (SHA-1)	0.32 ms	36 μs	18 μs	0.23 μs
$T_{SE/D}$ (GCM)	5.6 ms	0.48/1.15 ms	0.24/0.57 ms	2.0/6.9 μs
T_{ECM} (P-256)	17.1 ms	276 ms	138 ms	1.75 ms
HKDF-SHA256	–	629 μs	315 μs	3.2 μs
T_{ECM}/T_H	53	7,600	7,600	7,700

Source: `research/paper_audit/results/*.csv`. All benchmark outputs are checksummed: the 41 digests are identical on all BearSSL platforms. On the ESP8266 the 25th and 75th percentiles equal the median, and a rerun on a second board (6 October 2026) reproduced every median exactly. On the laptop, absolute times shifted by 1.3–2.3$\times$ in Low Power Mode, so only ratios are used from it.

Protocol-level checks (re-run)

- **Forward secrecy:** an attacker who later obtains $k^\star$ recovered 5 of 5 past session keys from recorded M_1/M_2 alone.
- **Server lookup:** resolving a pseudonym took 0.47, 4.81 and 23.85 ms on the Apple-silicon laptop at 100, 1,000 and 5,000 devices, using a 13-epoch window (committed result). On the Linux laptop the same script measured 1.80, 19.86 and 77.57 ms at 100, 1,000 and 5,000 devices. In the AWS test, server cost grew by about 13 μs per enrolled device, reaching 26.1 ms at 1,800 devices.

4.4 Result Analysis

Implementation objectives. All five implementation objectives were met: 97% coverage against a 90% target; 10,000 correctness trials with no failures; a working three-terminal demo with a correctly classified replay rejection; all eight SEC cases passing; and a median authentication time of about 0.1 ms, against a 5 ms budget. The measurement objectives were also met: the device side runs on an ESP8266 and interoperates with the Python server, and the primitives were measured on three hardware classes.

Consistency of the paper's evidence. Table 4.9 summarises the audit (re-run with `audit.py`, `figures_audit.py` and `fig1_repro.py`; outputs match the committed results).

Table 4.9: Consistency audit of the base paper

#	**Finding**	**Evidence**
A1	17/21/12% not derivable	None of nine definitions gives these values within one point. Table IV: 36.0% vs best, 57.5% vs mean. Table VI: 10.1% (device) and 33.3% (server) vs best. Figures: 24–49%, 19–31%, 7–25%.
A2	Tables contradict text	Table IV: 4 messages, 1,024 bits. Text and Eq. 21: 2 messages, 264 bits, yet $2 \times 128 = 256$, and a SHA-256 pseudonym plus a GCM tag already exceed 264 bits in M_1 alone.
A3	Not the cheapest under its own constants	Table V $\times$ Table III: proposed 23.68 ms (device + server); Turkanović 3.84, Dhillon–Kalra 4.48, "Jiang" 4.80 ms.
A4	Internal inconsistencies	Table V charges $2T_H + 2T_{SE/D}$, Eq. 22 one KDF, one encryption and one MAC. Algorithms 1–2 describe a different protocol. Table VI is captioned "communication". Two schemes share reference [17]. "5% (one per hundred devices)".
A5	Figures disagree with text	Fig. 1 captioned processing, axis communication. Figs. 1–2 plot "vehicles in VANET". Fig. 2's per-session bit cost rises with vehicles. The figures compare against four schemes cited nowhere in the paper.
A6	Row labels do not match references	"Cheng and Le [14]", "Jiang [15]", "Shuai [16]" point to Wang *et al.*, Wen *et al.* and the chaotic-map Jiang paper. "Cheng–Le" matches Chang and Le's P2 with counts exchanged. Turkanović's device and server counts are swapped.
A7	Constants differ from source papers	T_C: 0.32 ms in the base paper vs 32.40 ms in Jiang *et al.*; T_{ECM}: 17.1 vs 63.075 ms in Chang and Le.

Fig. 1 could not be reproduced. The paper gives no simulation scenario, code or data, and two web searches found none, so the simulation is *not publicly available*. The paper's own cost method gives a per-authentication cost independent of the number of vehicles, but the digitised Fig. 1 curves rise by 5–21 units. A flat fit leaves residuals of 10–36 times the reading error, and even an affine fit leaves the proposed curve at 20 times. This shows only that Fig. 1 depends on undocumented simulation effects, not that it is wrong. The team's own OMNeT++ model of the Table II setting (Figure 4.2) depends on guessed inputs (adversary rate, arrival window, which devices a server must resolve, server class). The proposed scheme's mean latency at 1,000 devices ranges

from about 2 ms to 5.6 s across those guesses, so the model is used only as a sensitivity analysis.

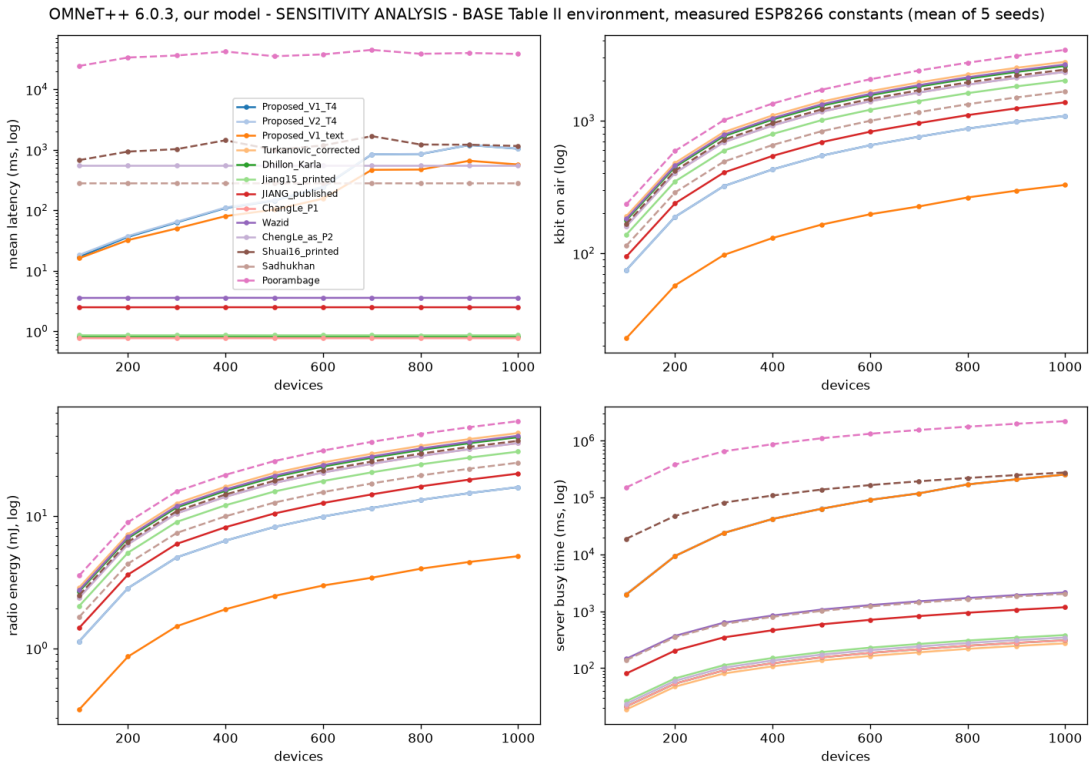


Figure 4.2: OMNeT++ sensitivity analysis of the base paper's Table II setting with measured ESP8266 constants (team's model; mean of 5 seeds)

Recomputed ranking against ECC-based schemes. With measured ESP8266 constants (80 MHz, device + server, `aes_big`/`aes_ct`), the proposed scheme costs 2.1/4.7 ms. Sadhukhan, Shuai and "Cheng–Le" cost 279–553 ms, Challa 2,487 ms and Porambage 5,251 ms, so the proposed scheme is about 60 to 2,500 times cheaper. Because a P-256 multiplication is 16 times slower than the paper assumes, the advantage over ECC schemes is far larger than the paper reports.

Recomputed ranking against hash-only schemes. Table 4.10 counts how many of the three hash-only rows (Turkanović, Dhillon–Kalra, "Jiang") cost less than the proposed scheme. Measured HKDF costs 6–17 T_H on every platform, more than the hash-only schemes' entire device columns (5–7 T_H). Once the session KDF the paper's own equations require is counted (V2–V4), all three hash-only rows are cheaper in every column on every platform, including OpenSSL with hardware AES and SHA. On the ESP8266 the gap is 3.8–13.7×. The conclusion is firm for Turkanović and Dhillon–Kalra, whose counts are confirmed by independent papers. For "Jiang" it holds to the extent that the survey counts for the two-factor scheme are right. The chaotic-map scheme of Jiang *et al.*, with its own $T_C = 32.40$ ms, costs about 195 ms on the ESP8266 and is far dearer than the proposed scheme.

Table 4.10: Hash-only rows of Table V cheaper than the proposed scheme (device + server; GCM, cached key, `aes_big`; re-run with `eq22_variants.py`)

Platform	**V1**	**V2**	**V3**	**V4**
ESP8266, 80 and 160 MHz (BearSSL)	3/3	3/3	3/3	3/3
Graviton2 t4g.nano (BearSSL)	3/3	3/3	3/3	3/3
Laptop, BearSSL software	3/3	3/3	3/3	3/3
Laptop, OpenSSL hardware AES/SHA	1/3	3/3	3/3	3/3

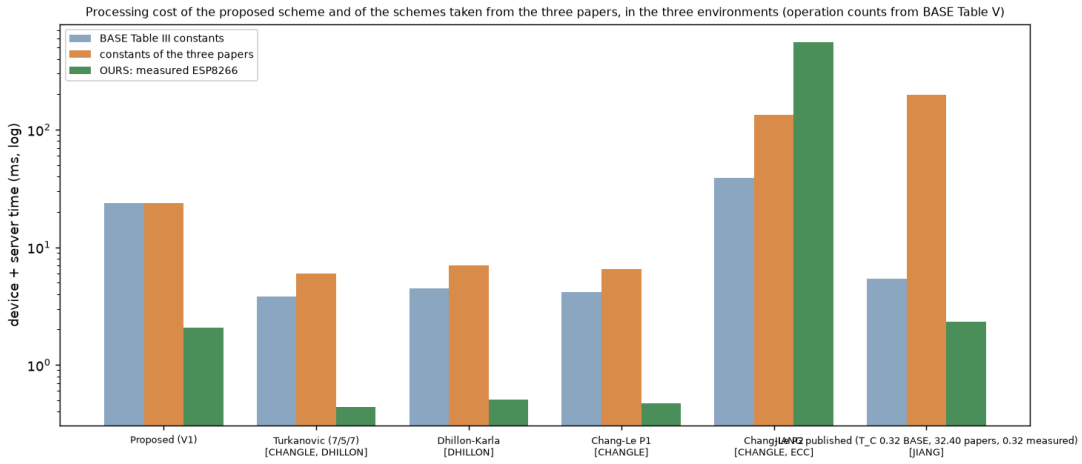


Figure 4.3: Processing cost (device + server, log scale) of the proposed scheme and the comparator schemes under the base paper's constants, the comparator papers' own constants, and measured ESP8266 constants (team's replication)

Communication and storage. Against the paper's reported values, the measured 182 bytes of protocol fields is 39.6% below the mean of Table IV's baselines, and the team's network model also gives the proposed scheme the fewest bits on air in every variant. The direction of the communication claim therefore holds, but the magnitude cannot be checked, because the paper's own figures disagree (1,024 vs 264 bits). Table VI's storage arithmetic reproduces (10.1% and 33.3%), but not 12%. The 320-bit server storage also holds only if the server keeps no per-device pseudonym table; a precomputed table at 1,000 devices needs about 416 KB.

Laptop measurement vs. the paper. The benchmark report's “99.9% processing reduction” compares 0.1 ms measured on a laptop with the paper's embedded-hardware constants. It is not meaningful and is not used as a result. The ratio-based recomputation above replaces it.

4.5 Observation/Remarks

- **The protocol itself is light.** On an ESP8266 a whole handshake costs 5.26 ms of computation, about 52 times less than a single P-256 multiplication on the same

chip. The paper's weakness is its evidence and constants, not the design idea.

- **ECC is nominal.** λ_X is a hash reduced modulo n; the public point is optional and unused by Eq. 9–22. No elliptic-curve operation occurs online.
- **No forward secrecy.** Compromise of $k^\star$ exposes every past session key, unlike ECC schemes such as Chang and Le's P2.
- **Server cost is $O(\textbf{devices} \times \textbf{window})$.** Unlinkable pseudonyms cannot be indexed, so the server probes every device's epoch window. The paper's Table V treats server cost as constant.
- **Weaknesses of the implementation found in the field test.** (i) The device epoch lives only in memory: a restarted device is rejected as a replay, or locked out after many sessions (1,020 of 1,020 retries failed after 2,020 sessions). The ESP8266 firmware avoids this by persisting the epoch to flash. (ii) `hea-ta enroll` distributes keys as a full mesh, so one device's credential file was enough to impersonate another device (`impersonate.py` succeeded). (iii) The block-list and replay window are lost on server restart: a blocked device authenticated again, and a pre-restart M_1 was accepted once (without yielding a session key). These are flaws in this implementation, not in the paper.
- **The operation-count model under-predicts real cost.** It gives 2.4 ms of device computation on the ESP8266, against 5.26 ms measured, because it omits HKDF, JSON/base64 parsing and per-message key scheduling.
- **A transcription error remains in the benchmark code.** `bench/baselines/reference_data.py` still lists the proposed scheme's server cost as $4T_H + 5T_{ECM}$ (should be $2T_H + 2T_{SE/D}$) and Sadhukhan's server cost as $2T_H + 4T_{ECM}$ (should be $2T_H + 4T_{SE/D}$). The audit scripts use the correct values in `research/paper_audit/paper_tables.py`.
- **Earlier claims were corrected.** An `openssl speed` proxy that suggested hardware crypto would reverse the ranking was withdrawn as not like-for-like. The team's replication also corrected the manuscript's range against ECC schemes from "120–3,000×" to 60–2,520×.

4.6 Conclusion

The base paper's Section III protocol was implemented in Python and, on the device side, in C for an ESP8266, and the two interoperate. The implementation passes all 108 tests, runs between two cloud instances at 0.73 ms per handshake, and costs 5.26 ms

of computation on a microcontroller. Using this implementation and a cross-platform primitive benchmark, the project found that: the paper's headline percentages cannot be derived from its own data; its Fig. 1 cannot be reproduced from published information; its assumed constants are off by 16× (ECC) and 5–12× (symmetric) on a real microcontroller; the scheme is 60–2,500× cheaper than ECC-based schemes but more expensive than the verified hash-only schemes once its own session KDF is counted; and it lacks forward secrecy and has a server lookup cost that grows with the number of devices. The field test also exposed three implementation weaknesses that a deployment would have to fix. Chapter 5 summarises the limitations and future work.

CHAPTER: 5

CONCLUSION

CHAPTER 5

CONCLUSION

5.1 Limitations

- **Not production software.** There is no HSM or secure element, no formal verification (ProVerif or Tamarin), no penetration test and no side-channel hardening. The key store relies on file permissions only.
- **Implementation weaknesses not yet fixed.** The Python device epoch is not persisted. TA enrolment distributes keys as a full mesh, which allows device-to-device impersonation. The server's block-list and replay window do not survive a restart. Pseudonym lookup is linear in the number of devices.
- **No forward secrecy, static devices, single TA.** These limitations are inherited from the base scheme. Rekeying (Eq. 24) and revocation are not implemented.
- **One microcontroller class.** Only a 32-bit ESP8266 was measured. There is no 8- or 16-bit part, no MCU with AES or SHA hardware, and no energy measurement.
- **One library family.** Primitive costs are BearSSL's and OpenSSL's; other libraries, or a hardware HKDF/HMAC engine, would shift the constants.
- **Competitors modelled, not run.** Comparator costs are the base paper's operation counts multiplied by measured times. The “Jiang” and “Shuai” rows were traced only through secondary tables, and no storage entry could be checked. T_C and T_{fe} were not measured.
- **Simulation is a sensitivity analysis.** The OMNeT++ model is the team's own and uses guessed inputs. It says nothing definitive about the base paper's figures.
- **Transcription limits.** Tables were transcribed from rendered PDFs (Table V by hand), and figure values were read by eye. The authors of the base paper were not contacted.

5.2 Conclusion

This project set out to turn a published but unimplemented hybrid ECC mutual authentication protocol for IoT into working software, and then to test the paper's claims against it. The protocol was implemented in Python following the paper's secure, nonce-based Section III model, and its device side was ported to C and run on an ESP8266

against the unmodified Python server. All implementation objectives were met: 108 passing tests with 97% coverage, 10,000 correct key agreements, all eight adversarial cases, and working demonstrations on a laptop, two cloud nodes and a microcontroller. Measuring the primitives on real hardware showed that the paper's cost model rests on constants that are wrong in both directions. An ECC scalar multiplication is sixteen times slower than assumed, and symmetric operations are five to twelve times faster. As a result, the scheme beats ECC-based schemes by far more than the paper claims, but loses to verified hash-only schemes once the session-key derivation required by its own equations is counted. The paper's headline percentages cannot be derived from its own data, and its simulation cannot be reproduced from published information. The scheme also lacks forward secrecy and has a server lookup cost that grows with the number of devices. The protocol design is light and practical; what the project corrects is the evidence used to rank it.

5.3 Future Scope

- Obtain the base paper's simulation scenario from its authors and rerun Fig. 1 against it.
- Measure an MCU with hardware AES and SHA, an 8/16-bit part and a Raspberry Pi 3, including energy.
- Read the original Shuai *et al.* and two-factor Jiang *et al.* papers to confirm the secondary tabulations, and verify the remaining Table IV–VI entries.
- Implement and time Turkanović *et al.* end to end on the microcontroller instead of modelling it.
- Fix the implementation weaknesses: persist the device epoch, give devices only server secrets, persist the server's block-list and replay window, and replace the linear pseudonym scan with an index of expected next pseudonyms.
- Complete the TA-assisted rekeying and key-revocation extension; consider an optional ephemeral key exchange for forward secrecy and measure its cost (one P-256 multiplication is 276 ms on the ESP8266).
- Model the Section III message flow in Tamarin or ProVerif.
- Correct the transcription error in `bench/baselines/reference_data.py`.

BIBLIOGRAPHY

[1]A. Al-Rasheed, R. Khan, A. Ahmad, A. Saeed, F. Alturise, and S. Alkhalaf, "A low-cost hybrid elliptic curve cryptography authentication protocol for trustworthy Internet of Things communication," *IEEE Transactions on Consumer Electronics*, vol. 72, pp. 4483–4491, May 2026.

[2]J. Kerkar, C. Kelkar, C. Poornamath, Y. Kolekar, and S. Sondur, "Do the numbers hold on real hardware? A reproducibility and measurement audit of a low-cost hybrid ECC authentication protocol for IoT." Manuscript, release tag `paper-v1`, `https://github.com/jaykerkar0405/hybrid-ecc-iot-auth`, 2026.

[3]C.-C. Chang and H.-D. Le, "A provably secure, efficient, and flexible authentication scheme for ad hoc wireless sensor networks," *IEEE Transactions on Wireless Communications*, vol. 15, pp. 357–366, Jan. 2016.

[4]M. Turkanović, B. Brumen, and M. Hölbl, "A novel user authentication and key agreement scheme for heterogeneous ad hoc wireless sensor networks, based on the Internet of Things notion," *Ad Hoc Networks*, vol. 20, pp. 96–112, Sept. 2014.

[5]Q. Jiang, F. Wei, S. Fu, J. Ma, G. Li, and A. Alelaiwi, "Robust extended chaotic maps-based three-factor authentication scheme preserving biometric template privacy," *Nonlinear Dynamics*, vol. 83, pp. 2085–2101, Mar. 2016.

[6]Python Cryptographic Authority, "cryptography – Python library documentation." `https://cryptography.io`. Version 50.0.0 used in this project.

[7]T. Pornin, "BearSSL, a smaller SSL/TLS library." `https://bearssl.org`. The ESP8266 Arduino core ships a port of BearSSL.

[8]P. K. Dhillon and S. Kalra, "A lightweight biometrics based remote user authentication scheme for IoT services," *Journal of Information Security and Applications*, vol. 34, pp. 255–270, 2017.

[9]J. Zhong, S. He, Z. Liu, and L. Xiong, "Lightweight anonymous authentication for IoT: A taxonomy and survey of security frameworks," *Sensors*, vol. 25, p. 5594, Sept. 2025.

[10]Q. Jiang, J. Ma, X. Lu, and Y. Tian, "An efficient two-factor user authentication scheme with unlinkability for wireless sensor networks," *Peer-to-Peer Networking and Applications*, vol. 8, no. 6, pp. 1070–1081, 2015.

[11]J. Zong, C. Wang, J. Shen, C. Su, and W. Wang, "ReLAC: Revocable and lightweight access control with blockchain for smart consumer electronics," *IEEE Transactions on Consumer Electronics*, vol. 70, pp. 3994–4004, Feb. 2024.

[12]M. Nawaz Khan, H. Ur Rahman, T. Hussain, B. Yang, and S. Mian Qaisar, "Enabling trust in automotive IoT: Lightweight mutual authentication scheme for electronic connected devices in Internet of Things," *IEEE Transactions on Consumer Electronics*, vol. 70, pp. 5065–5078, Aug. 2024.

[13]B. Gong, G. Zheng, M. Waqas, S. Tu, and S. Chen, "LCDMA: Lightweight cross-domain mutual identity authentication scheme for Internet of Things," *IEEE Internet of Things Journal*, vol. 10, pp. 12590–12602, July 2023.

[14]I. Alshawish and A. Al-Haj, "An efficient mutual authentication scheme for IoT systems," *The Journal of Supercomputing*, vol. 78, pp. 16056–16087, Sept. 2022.

[15]S. Qiu, J. Li, X. Di, X. Li, Y. Wu, and M. Ibrahim, "Lightweight mutual authentication scheme based on blockchain for Internet of Medical Things," *IEEE Internet of Things Journal*, vol. 12, pp. 8848–8861, Apr. 2025.

[16]D. Yin and B. Gong, "A lightweight certificateless mutual authentication scheme based on signatures for IIoT," *IEEE Internet of Things Journal*, vol. 11, pp. 26852–26865, Aug. 2024.

[17]N. Sharma and P. Dhiman, "A secure addressing mutual authentication scheme for smart IoT home network," *Multimedia Tools and Applications*, vol. 84, pp. 25111–25143, Aug. 2024.

[18]P. Porambage, C. Schmitt, P. Kumar, A. Gurtov, and M. Ylianttila, "Two-phase authentication protocol for wireless sensor networks in distributed IoT applications," in *Proc. IEEE Wireless Communications and Networking Conference (WCNC)*, pp. 2728–2733, Apr. 2014.

[19]D. Sadhukhan, S. Ray, G. P. Biswas, M. K. Khan, and M. Dasgupta, "A lightweight remote user authentication scheme for IoT communication using elliptic curve cryptography," *The Journal of Supercomputing*, vol. 77, pp. 1114–1151, Feb. 2021.

[20]S. Challa *et al.*, "Secure signature-based authenticated key establishment scheme for future IoT applications," *IEEE Access*, vol. 5, pp. 3028–3043, 2017.

[21]M. Fakroon, M. Alshahrani, F. Gebali, and I. Traore, "Secure remote anonymous user authentication scheme for smart home environment," *Internet of Things*, vol. 9, p. 100158, Mar. 2020.

[22]S. Banerjee, V. Odelu, A. K. Das, S. Chattopadhyay, and Y. Park, “An efficient, anonymous and robust authentication scheme for smart home environments,” *Sensors*, vol. 20, p. 1215, Feb. 2020.

[23]M. Shuai, N. Yu, H. Wang, and L. Xiong, “Anonymous authentication scheme for smart home environment with provable security,” *Computers & Security*, vol. 86, pp. 132–146, 2019.

[24]H. Krawczyk and P. Eronen, “HMAC-based extract-and-expand key derivation function (HKDF).” RFC 5869, Internet Engineering Task Force, 2010.

[25]M. Dworkin, “Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC,” Tech. Rep. SP 800-38D, National Institute of Standards and Technology, 2007.

[26]OpenSSL Project, “OpenSSL cryptography and SSL/TLS toolkit.” `https://www.openssl.org`.

[27]OpenSim Ltd., “OMNeT++ discrete event simulator, version 6.0.3.” `https://omnetpp.org`.

APPENDIX

1. Paper Details

The team has written an IEEE conference-format manuscript from this work (`paper/main.tex`, release tag `paper-v1`, 26 October 2026) [2].
Title: Do the Numbers Hold on Real Hardware? A Reproducibility and Measurement Audit of a Low-Cost Hybrid ECC Authentication Protocol for IoT
Authors: Jay Kerkar, Chaitanya Kelkar, Chirag Poornamath, Yash Kolekar; Dr. Shanta Sondur (Project Guide), Department of Information Technology, VESIT, Mumbai
Code and data: `https://github.com/jaykerkar0405/hybrid-ecc-iot-auth`

- **Paper published:** **[MISSING: submission status, venue/conference, acceptance or publication date, DOI/link – the repository records only the manuscript]**
- **Certificate of publication:** **[MISSING: attach if applicable]**
- **Plagiarism report:** **[MISSING: attach the department-required report]**

2. Competition certificates

[MISSING: No competition participation or certificates are recorded in the repository. Attach if applicable, or confirm this section should state “Not applicable”.]